final-melhorado-v2

June 15, 2026

[1]:
```
'''
Bibliotecas usadas
'''

import os
from datetime import datetime
import re
import subprocess
import warnings
import numpy as np
import pandas as pd
from collections import Counter
from Bio import Entrez
from Bio import SeqIO
from Bio.Seq import Seq
from sklearn.model_selection import (
    GroupShuffleSplit,
    StratifiedGroupKFold,
    RandomizedSearchCV,
    GroupKFold
    )
from sklearn.metrics import (
    classification_report,
    roc_auc_score,
    average_precision_score,
    make_scorer,
    matthews_corrcoef,
    f1_score,
    precision_score,
    recall_score,
    balanced_accuracy_score,
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    auc
)
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_validate
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
import subprocess
import shap
import seaborn as sns
from sklearn.inspection import permutation_importance


warnings.filterwarnings("ignore")

print("Bibliotecas importadas")
```

```
Bibliotecas importadas
```

[2]:
```
'''
obtenção dos datasets
'''

EMAIL = "marcela.leite@gmail.com.br"
OUTPUT_DIR = "pipelineAE"
os.makedirs(OUTPUT_DIR, exist_ok=True)
Entrez.email = EMAIL

# ============================================================
# QUERIES
# ============================================================

POSITIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum betaceum"[Organism] OR "Solanum␣
 ↪melongena"[Organism] OR "Solanum tuberosum"[Organism] OR "Physalis␣
 ↪peruviana"[Organism]))
    AND (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA␣
 ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "Rx1" OR "Sw-5" OR "Tm-2" OR "Tm-22" OR "RCY1" OR "Rx" OR "N GENE"
    OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR" OR "CC-NBS-LRR" OR "TNL"␣
 ↪OR "CNL Domain" OR "CNL"
    OR "NBS-ARC" OR "NB-ARC domain" OR "NB-ARC" OR "Nucleotide-binding site"
    OR "RDR" OR "Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" )
    NOT (partial OR fragment OR hypothetical OR predicted OR DAP-seq OR␣
 ↪"transcription factor" OR DNA-binding OR microtom)
```

```
    NOT ("pathogenesis-reletad protein" OR osmotin OR PR1 OR PR2 OR PR3 OR PR4␣
 ↪OR PR5 OR synthase OR metabolic OR precursor)
    NOT txid10239[Organism:exp]
'''
NEGATIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum melongena"[Organism] OR "Solanum␣
 ↪tuberosum"[Organism] OR "Physalis peruviana"[Organism]))
   AND 100:3000[Sequence Length]
   NOT (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA␣
 ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "Rx1" OR "Sw-5" OR "Tm-2" OR "Tm-22" OR "RCY1" OR "Rx" OR "N"
    OR "virus resistance" OR "antiviral" OR "Tobacco mosaic virus" OR "Potato␣
 ↪virus"
    OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR" OR "CC-NBS-LRR" OR "TNL"␣
 ↪OR "CNL Domain" OR "CNL"
    OR "NBS-ARC" OR "NB-ARC domain" OR "NB-ARC" OR "Nucleotide-binding site"
    OR "RNA silencing" OR "RDR" OR "Dicer-like" OR "Argonaute" OR "AGO" OR␣
 ↪"eIF4E" )
   NOT (partial OR fragment OR hypothetical OR predicted OR uncharacterized OR␣
 ↪DAP-seq OR "transcription factor" OR DNA-binding OR microtom )
'''

ARABIDOPSIS_QUERY = '''
   "Arabidopsis thaliana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
'''

SOLANUM_MELONGENA = '''
    "Solanum melongena"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
    '''

TUBEROSUM_QUERY = '''
    "Solanum tuberosum"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
    '''

PHYSALIS_QUERY = '''
  "Physalis peruviana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
      '''

# ============================================================
# DOWNLOAD NCBI
# ============================================================
```

```python
def baixar_proteinas_ncbi(query, output_fasta, max_records=35000):
    print("\n================================")
    print("DOWNLOAD NCBI: ",output_fasta)
    print("================================")
    handle = Entrez.esearch(
        db="protein",
        term=query,
        retmax=max_records
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Proteínas encontradas: {len(ids)}")
    if len(ids) == 0:
        return
    handle = Entrez.efetch(
        db="protein",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

def baixar_transcriptoma(output_fasta, query):
    print("\n================================")
    print("DOWNLOAD TRANSCRIPTOMA: ",output_fasta)
    print("================================")
    handle = Entrez.esearch(
        db="nucleotide",
        term=query,
        retmax=50000
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Transcritos encontrados: {len(ids)}")
    handle = Entrez.efetch(
        db="nucleotide",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

#Dados de Entrada - baixar arquivos FASTA das sequências de proteínas
```

```
POSITIVE_FASTA = f"{OUTPUT_DIR}/positive.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative.fasta"


print("\n============ FINALIZADO CONFIGURAÇÃO DE PARÂMETROS ============\n")
```

```
============ FINALIZADO CONFIGURAÇÃO DE PARÂMETROS ============
```

[3]:
```
inicio = datetime.now()
print("\n\nInciando download da classe positiva: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(POSITIVE_QUERY,POSITIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download da classe positiva:  12:12:10

================================
DOWNLOAD NCBI:  pipelineAE/positive.fasta
================================
Proteínas encontradas: 8914
Arquivo salvo: pipelineAE/positive.fasta


Finalizado em: 12:13:01

Tempo decorrido: 0:00:51.385897
```

[4]:
```
inicio = datetime.now()
print("\n\nInciando download da classe negativa: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(NEGATIVE_QUERY,NEGATIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\n\nArquivos salvos na pasta: ",OUTPUT_DIR)
print("============ FIM COLETA DE DADOS TREINO/VALIDAÇÃO ============")
```

```
Inciando download da classe negativa:  12:13:01

================================
DOWNLOAD NCBI:  pipelineAE/negative.fasta
================================
```

```
Proteínas encontradas: 35000
Arquivo salvo: pipelineAE/negative.fasta


Finalizado em: 12:14:04

Tempo decorrido: 0:01:02.563201


Arquivos salvos na pasta:  pipelineAE
============ FIM COLETA DE DADOS TREINO/VALIDAÇÃO ============
```

[5]:
```
#Dados para aplicação - baixar transcriptomas - RNA-Seq
transcriptoma_arabidopsis_fasta = (f"{OUTPUT_DIR}/arabidopsis_transcriptoma.
 ↪fasta")
transcriptoma_melongena_fasta = (f"{OUTPUT_DIR}/melongena_transcriptoma.fasta")
transcriptoma_tuberosum_fasta = (f"{OUTPUT_DIR}/tuberosum_transcriptoma.fasta")
transcriptoma_physalis_fasta = (f"{OUTPUT_DIR}/physalis_transcriptoma.fasta")
```

[6]:
```
print("\n\nInciando downaload de dados de transcriptmas")
inicio = datetime.now()
print("\n\nInciando download ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_arabidopsis_fasta, ARABIDOPSIS_QUERY)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando downaload de dados de transcriptmas


Inciando download ARABIDOPSIS:  12:14:04

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAE/arabidopsis_transcriptoma.fasta
================================
Transcritos encontrados: 49951
Arquivo salvo: pipelineAE/arabidopsis_transcriptoma.fasta


Finalizado em: 12:14:51

Tempo decorrido: 0:00:47.657552
```

[7]:
```
inicio = datetime.now()
print("\n\nInciando download PHYSALIS: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_physalis_fasta, PHYSALIS_QUERY)
```

```
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download PHYSALIS:  12:14:51

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAE/physalis_transcriptoma.fasta
================================
Transcritos encontrados: 21702
Arquivo salvo: pipelineAE/physalis_transcriptoma.fasta


Finalizado em: 12:15:31

Tempo decorrido: 0:00:39.376683
```

[8]:
```
inicio = datetime.now()
print("\n\nInciando download MELONGENA: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_melongena_fasta, SOLANUM_MELONGENA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download MELONGENA:  12:15:31

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAE/melongena_transcriptoma.fasta
================================
Transcritos encontrados: 8555
Arquivo salvo: pipelineAE/melongena_transcriptoma.fasta


Finalizado em: 12:16:08

Tempo decorrido: 0:00:37.349548
```

[9]:
```
inicio = datetime.now()
print("\n\nInciando download TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_tuberosum_fasta, TUBEROSUM_QUERY)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
print("\n\n\nArquivos salvos na pasta: ",OUTPUT_DIR)
print("============ FIM COLETA DE DADOS APLICAÇÃO ============")
```

```
Inciando download TUBEROSUM:  12:16:08

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAE/tuberosum_transcriptoma.fasta
================================
Transcritos encontrados: 20832
Arquivo salvo: pipelineAE/tuberosum_transcriptoma.fasta


Finalizado em: 12:16:50

Tempo decorrido: 0:00:42.352060



Arquivos salvos na pasta:  pipelineAE
============ FIM COLETA DE DADOS APLICAÇÃO ============
```

[10]:
```
'''
Tratamento dos dados
    - Criar Dataframes com os conjuntos de dados de treino e validação
    - Executar CD-Hit para eliminar sequencias muitos semelhantes
    - Extração de atributos (features)

'''

# ============================================================
# AAC
# ============================================================
# composição de aminoácidos - transformar a proteína em dados numéricos
# conta a frequência de cada aminoácido em uma sequência de proteína
# como são 20 aminoácidos possíveis, irá gerar 20 features com seus percentuais
# com essa frequência consegue ter um parâmetro para distinguir as proteínas
# por exemplo proteínas de resistência tem geralmente certa frequência de␣
 ↪aminoácidos
# mas ignora a ordem - por isso a sequência de dipeptídeos irá incrementar o␣
 ↪modelo considerando a ordem
# sequência de 20 aminoácidos
AMINO_ACIDOS = list("ACDEFGHIKLMNPQRSTVWY")

motifs = {
    # ==========================
```

```
    # NLR (CNL/TNL)
    # ==========================
    "P_LOOP": r"[AGST].{3,5}GK[TS]",
    "KINASE2": r"[ALIVMFYW]{3,5}DD[VILW][WFY]",
    "GLPL": r"G[LVI][PAST][LIVAF]",
    "MHD": r"[MVIL][HN][DE][VILAST]",
    "RNBS_A": r"F.DL.{0,3}AW",
    "RNBS_B": r"G[SA].{2,6}T",
    "RNBS_C": r"Y.[VIL].{1,3}[LS]",
    "RNBS_D": r"C.{2,8}P",

    # ==========================
    # Protein Kinase (KIN/RLK)
    # ==========================
    "HRD": r"HRD[LIVMFY]",
    "DFG": r"DFG",

    # ==========================
    # LRR (RLP/RLK)
    # ==========================
    "LRR1": r"L..L..L..N",
    "LRR2": r"L[A-Z]{2}L[A-Z]{2}L[A-Z]{2}[NCT]"

}

def calcular_aac(seq):
    seq = seq.upper()
    total = len(seq)
    if total == 0:
        return [0] * len(AMINO_ACIDOS)
    counts = Counter(seq)
    return [
        counts.get(aa, 0) / total
        for aa in AMINO_ACIDOS
    ]

# ============================================================
# DIPEPTÍDEOS
# ============================================================
'''
encontrar pares de aminoácidos consectíveis em uma sequência protéica ␣
 ↪-sequência de dois aminoácidos
considera a ordem dos aminoácidos
há certos padrões comuns para
para 20 aminoácidos, há possibilidade de 20*20 = 400 dipeptídeos - features
'''
```

```
DIPEPTIDES = [
    a + b
    for a in AMINO_ACIDOS
    for b in AMINO_ACIDOS
]

def calcular_dipeptideos(seq):
    seq = seq.upper()
    total = len(seq) - 1
    if total <= 0:
        return [0] * len(DIPEPTIDES)
    counts = Counter([
        seq[i:i+2]
        for i in range(total)
    ])
    return [
        counts.get(dp, 0) / total
        for dp in DIPEPTIDES
    ]

def localizar_motivos(seq):
    posicoes = {}
    for nome, pattern in motifs.items():
        match = re.search(pattern, seq)

        if match:
            posicoes[nome] = match.start()
        else:
            posicoes[nome] = None

    return posicoes


def calcular_distancias(posicoes):

    def distancia(a, b):
        if posicoes[a] is None or posicoes[b] is None:
            return -1
        return posicoes[b] - posicoes[a]

    return {
        "dist_ploop_kinase2": distancia("P_LOOP", "KINASE2"),
        "dist_kinase2_glpl": distancia("KINASE2", "GLPL"),
        "dist_glpl_mhd": distancia("GLPL", "MHD")
    }

def contar_motivos(posicoes):
```

```
    return sum(
        1 for v in posicoes.values()
        if v is not None
    )

def extrair_motifs(seq):

    return [
        int(bool(re.search(pattern, seq)))
        for pattern in motifs.values()
    ]



# ============================================================
# FEATURES - atributos
# ============================================================
#calculando features
def extrair_features_proteina(seq):
    seq = re.sub(r"[^ACDEFGHIKLMNPQRSTVWY]", "", seq)
    if len(seq) < 50:
        return None
    features = []
    features.append(len(seq)) # 1 - comprimento da cadeia
    features.extend(calcular_aac(seq)) # 20 composição de aminoácidos - frequência de cada um na sequência
    features.extend(calcular_dipeptideos(seq)) #400 possibilidades - considera a ordem dos aminoácidos para buscar os mais frequêntes em proteínas de resistência

    features.extend(extrair_motifs(seq))
    # posições dos motivos
    posicoes = localizar_motivos(seq)
    # número de motivos encontrados
    num_motifs_detectados = contar_motivos(posicoes)
    # distâncias entre motivos
    distancias = calcular_distancias(posicoes)

    features.append(num_motifs_detectados)
    protein_length = len(seq)

    features.append(distancias["dist_ploop_kinase2"])
    features.append(distancias["dist_kinase2_glpl"])
    features.append(distancias["dist_glpl_mhd"])

    features.append(
        distancias["dist_ploop_kinase2"]/protein_length
```

```
        if distancias["dist_ploop_kinase2"] != -1 else -1
    )
    features.append(
        distancias["dist_kinase2_glpl"]/protein_length
        if distancias["dist_kinase2_glpl"] != -1 else -1
    )
    features.append(
        distancias["dist_glpl_mhd"]/protein_length
        if distancias["dist_glpl_mhd"] != -1 else -1
    )

    features.append(int(posicoes["P_LOOP"] is not None))
    features.append(int(posicoes["KINASE2"] is not None))
    features.append(int(posicoes["GLPL"] is not None))
    features.append(int(posicoes["MHD"] is not None))

    encontrados = {}
    for nome, regex in motifs.items():
        encontrados[nome] = int( re.search(regex, seq) is not None)

    # features.append(encontrados[nome])

    num_nlr = (
        encontrados["P_LOOP"] +
        encontrados["KINASE2"] +
        encontrados["GLPL"] +
        encontrados["MHD"]
    )

    # num_kinase = (
    #     encontrados["HRD"] +
    #     encontrados["DFG"]
    # )

    # num_lrr = (
    #     encontrados["LRR1"] +
    #     encontrados["LRR2"]
    # )

    features.extend([
        num_nlr,
        # num_kinase,
        # num_lrr
    ])

    return features
```

```
# ============================================================
# DATASET
# ============================================================
motif_cols = [
    "P_LOOP",
    "KINASE2",
    "GLPL",
    "MHD",
    "RNBS_A",
    "RNBS_B",
    "RNBS_C",
    "RNBS_D",
    "HRD",
    "DFG",
    "LRR1",
    "LRR2",
    "NUM_NLR_MOTIFS"
 ]
    # "NUM_KINASE_MOTIFS",
    # "NUM_LRR_MOTIFS"

def criar_dataset(positive_fasta, negative_fasta):
    data = []
    columns = (
        ["protein_length"]
        + [f"AAC_{aa}" for aa in AMINO_ACIDOS]
        + [f"DIPEP_{dp}" for dp in DIPEPTIDES]
        + motif_cols
        + ["num_motifs_detectados"]
        + ["dist_ploop_kinase2"]
        + ["dist_kinase2_glpl"]
        + ["dist_glpl_mhd"]
        + ["dist_ploop_kinase2_norm"]
        + ["dist_kinase2_glpl_norm"]
        + ["dist_glpl_mhd_norm"]
        + ["has_ploop"]
        + ["has_kinase2"]
        + ["has_glpl"]
        + ["has_mhd"]
    )

    # POSITIVOS
    for record in SeqIO.parse(positive_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
```

```
        row = feats + [1, record.id]
        data.append(row)

    # NEGATIVOS
    for record in SeqIO.parse(negative_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [0, record.id]
        data.append(row)

    df = pd.DataFrame(
        data,
        columns=columns + ["target", "id"]
    )
    print("\nTotal de Features/Atributos:",len(columns)) # quantidade
    print("\nFeatures/atributos:")
    print(columns)
    return df, columns

# CD-Hit
# chama o programa CD-HIT do sistema operacional
def executar_cdhit(
    input_fasta,
    output_fasta,
    identity=0.7
):
    cmd = [
        "cd-hit",
        "-i", input_fasta,
        "-o", output_fasta,
        "-c", str(identity),
        "-n", "5",
        "-M", "30000",
        "-T", "4"
    ]

    print("\nExecutando CD-HIT...")
    subprocess.run(cmd, check=True)
    print("CD-HIT concluído.")

def limpar(seq_id):
    seq_id = seq_id.strip()
    seq_id = seq_id.split()[0]
    return seq_id
```

```
def ler_clusters_cdhit(cluster_file):
    clusters = {}
    cluster_id = None
    with open(cluster_file) as f:
        for line in f:
            line = line.strip()
            if line.startswith(">Cluster"):
                cluster_id = line.replace(">Cluster ", "")
                clusters[cluster_id] = []
            else:
                seq_id = line.split(">")[1].split("...")[0]
                seq_id = limpar(seq_id)
                clusters[cluster_id].append(seq_id)
    return clusters

inicio = datetime.now()
print("\n\n\nInciando Tratamento dos dados: ", inicio.strftime("%H:%M:%S"))

# executa o CD-Hit nos arquivos baixados
print("\n\nInciar a execução do CD-HIT\n")
executar_cdhit(
    POSITIVE_FASTA,
    f"{OUTPUT_DIR}/positive_nr.fasta",
    identity=0.7
)

executar_cdhit(
    NEGATIVE_FASTA,
    f"{OUTPUT_DIR}/negative_nr.fasta",
    identity=0.7
)
# ========================================================
# Montar os DATASETs
# ========================================================

POSITIVE_FASTA = f"{OUTPUT_DIR}/positive_nr.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative_nr.fasta"


df, columns = criar_dataset(
    POSITIVE_FASTA,
    NEGATIVE_FASTA
)

df["id"] = df["id"].apply(limpar)
# ========================================================
# CLUSTERS
```

```
# ======================================================

clusters_pos = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/positive_nr.fasta.clstr"
)

clusters_neg = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/negative_nr.fasta.clstr"
)

cluster_map = {}

for c, seqs in clusters_pos.items():
    for s in seqs:
        cluster_map[s] = f"POS_{c}"

for c, seqs in clusters_neg.items():
    for s in seqs:
        cluster_map[s] = f"NEG_{c}"

df["cluster"] = df["id"].map(cluster_map)

print("\n Clusters ausentes: ")
print(df["cluster"].isna().sum())
df = df.dropna(subset=["cluster"])

print("\nDataset original pós CD-HIT:", df.shape)

# # ======================================================
# # NOVO BLOCO: BALANCEAMENTO IMPERATIVO (DOWNSAMPLING)
# # ======================================================
# print("\n--- Executando Balanceamento de Classes ---")
# df_positivos = df[df["target"] == 1]
# df_negativos = df[df["target"] == 0]

# print(f"Contagem inicial -> Positivos (R-genes): {len(df_positivos)} | Negativos (Background): {len(df_negativos)}")

# # Se houver mais negativos do que positivos (cenário padrão), reduzimos os negativos
# if len(df_negativos) > len(df_positivos):
#     # O sample() escolhe aleatoriamente N linhas do grupo majoritário
#     df_negativos_balanceados = df_negativos.sample(n=len(df_positivos), random_state=42)
#     df = pd.concat([df_positivos, df_negativos_balanceados]).reset_index(drop=True)
#     print(f"Subamostragem aplicada com sucesso nos Negativos.")
```

```python
# else:
#     print("O dataset já está balanceado ou possui mais positivos que negativos (incomum). Nenhuma ação foi tomada.")

# print(f"Contagem final   -> Positivos: {len(df[df['target'] == 1])} | Negativos: {len(df[df['target'] == 0])}")
# print("Dataset Final Pronto:", df.shape)


fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\n\nFim tratamento dos dados...")
```

```
Inciando Tratamento dos dados:  12:16:50


Inciar a execução do CD-HIT


Executando CD-HIT…
================================================================
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAE/positive.fasta -o
         pipelineAE/positive_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

Started: Mon Jun 15 12:16:50 2026
================================================================
                            Output
----------------------------------------------------------------
total seq: 8914
longest and shortest : 3540 and 51
Total letters: 7801441
Sequences have been sorted

Approximated minimal memory consumption:
Sequence        : 8M
Buffer          : 4 X 11M = 45M
Table           : 2 X 65M = 130M
Miscellaneous   : 0M
Total           : 185M

Table limit with the given memory limit:
Max number of representatives: 4000000
```

```
Max number of word counting entries: 3726820522

# comparing sequences from          0  to       1485
.---------- new table with      236 representatives
# comparing sequences from       1485  to       2723
100.0%---------- new table with      192 representatives
# comparing sequences from       2723  to       3754
----------     96 remaining sequences to the next cycle
---------- new table with      164 representatives
# comparing sequences from       3658  to       4534
99.8%---------- new table with      123 representatives
# comparing sequences from       4534  to       5264
100.0%---------- new table with       63 representatives
# comparing sequences from       5264  to       5872
…---------- new table with       88 representatives
# comparing sequences from       5872  to       6379
----------    105 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       6274  to       6714
----------     77 remaining sequences to the next cycle
---------- new table with      116 representatives
# comparing sequences from       6637  to       7016
----------     55 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       6961  to       7286
----------     58 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7228  to       7509
…---------- new table with       76 representatives
# comparing sequences from       7509  to       7743
…---------- new table with       66 representatives
# comparing sequences from       7743  to       7938
…---------- new table with       48 representatives
# comparing sequences from       7938  to       8914
…---------- new table with      247 representatives

     8914  finished       1719  clusters

Approximated maximum memory consumption: 190M
writing new database
writing clustering information
program completed !

Total CPU time 3.56
CD-HIT concluído.

Executando CD-HIT…
================================================================
```

```
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAE/negative.fasta -o
         pipelineAE/negative_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

Started: Mon Jun 15 12:16:52 2026
================================================================
                            Output
----------------------------------------------------------------
total seq: 9999
longest and shortest : 2993 and 100
Total letters: 4204131
Sequences have been sorted

Approximated minimal memory consumption:
Sequence        : 5M
Buffer          : 4 X 11M = 44M
Table           : 2 X 65M = 131M
Miscellaneous   : 0M
Total           : 181M

Table limit with the given memory limit:
Max number of representatives: 4000000
Max number of word counting entries: 3727298905

# comparing sequences from          0  to       1666
.---------- new table with     1374 representatives
# comparing sequences from       1666  to       3054
----------    547 remaining sequences to the next cycle
---------- new table with      694 representatives
# comparing sequences from       2507  to       3755
----------    796 remaining sequences to the next cycle
---------- new table with      264 representatives
# comparing sequences from       2959  to       4132
----------    771 remaining sequences to the next cycle
---------- new table with      296 representatives
# comparing sequences from       3361  to       4467
----------    729 remaining sequences to the next cycle
---------- new table with      311 representatives
# comparing sequences from       3738  to       4781
----------    694 remaining sequences to the next cycle
---------- new table with      290 representatives
# comparing sequences from       4087  to       5072
----------    640 remaining sequences to the next cycle
---------- new table with      247 representatives
# comparing sequences from       4432  to       5359
----------    628 remaining sequences to the next cycle
---------- new table with      256 representatives
# comparing sequences from       4731  to       5609
```

```
----------    565 remaining sequences to the next cycle
---------- new table with      204 representatives
# comparing sequences from       5044  to       5869
----------    546 remaining sequences to the next cycle
---------- new table with      213 representatives
# comparing sequences from       5323  to       6102
----------    531 remaining sequences to the next cycle
---------- new table with      199 representatives
# comparing sequences from       5571  to       6309
----------    499 remaining sequences to the next cycle
---------- new table with      150 representatives
# comparing sequences from       5810  to       6508
----------    451 remaining sequences to the next cycle
---------- new table with      197 representatives
# comparing sequences from       6057  to       6714
----------    427 remaining sequences to the next cycle
---------- new table with      176 representatives
# comparing sequences from       6287  to       6905
----------    411 remaining sequences to the next cycle
---------- new table with      134 representatives
# comparing sequences from       6494  to       7078
----------    361 remaining sequences to the next cycle
---------- new table with      153 representatives
# comparing sequences from       6717  to       7264
----------    323 remaining sequences to the next cycle
---------- new table with      147 representatives
# comparing sequences from       6941  to       7450
----------    298 remaining sequences to the next cycle
---------- new table with      133 representatives
# comparing sequences from       7152  to       7626
----------    280 remaining sequences to the next cycle
---------- new table with      142 representatives
# comparing sequences from       7346  to       7788
----------    250 remaining sequences to the next cycle
---------- new table with      125 representatives
# comparing sequences from       7538  to       7948
----------    246 remaining sequences to the next cycle
---------- new table with      107 representatives
# comparing sequences from       7702  to       8084
----------    217 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7867  to       8222
----------    211 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8011  to       8342
----------    181 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8161  to       8467
```

```
----------    172 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8295  to       8579
----------    131 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8448  to       8706
----------     22 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8684  to       8903
----------     68 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8835  to       9029
----------     47 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8982  to       9151
…---------- new table with       75 representatives
# comparing sequences from       9151  to       9999
…---------- new table with      405 representatives

     9999  finished       7092  clusters

Approximated maximum memory consumption: 194M
writing new database
writing clustering information
program completed !

Total CPU time 1.52
CD-HIT concluído.

Total de Features/Atributos: 445

Features/atributos:
['protein_length', 'AAC_A', 'AAC_C', 'AAC_D', 'AAC_E', 'AAC_F', 'AAC_G',
'AAC_H', 'AAC_I', 'AAC_K', 'AAC_L', 'AAC_M', 'AAC_N', 'AAC_P', 'AAC_Q', 'AAC_R',
'AAC_S', 'AAC_T', 'AAC_V', 'AAC_W', 'AAC_Y', 'DIPEP_AA', 'DIPEP_AC', 'DIPEP_AD',
'DIPEP_AE', 'DIPEP_AF', 'DIPEP_AG', 'DIPEP_AH', 'DIPEP_AI', 'DIPEP_AK',
'DIPEP_AL', 'DIPEP_AM', 'DIPEP_AN', 'DIPEP_AP', 'DIPEP_AQ', 'DIPEP_AR',
'DIPEP_AS', 'DIPEP_AT', 'DIPEP_AV', 'DIPEP_AW', 'DIPEP_AY', 'DIPEP_CA',
'DIPEP_CC', 'DIPEP_CD', 'DIPEP_CE', 'DIPEP_CF', 'DIPEP_CG', 'DIPEP_CH',
'DIPEP_CI', 'DIPEP_CK', 'DIPEP_CL', 'DIPEP_CM', 'DIPEP_CN', 'DIPEP_CP',
'DIPEP_CQ', 'DIPEP_CR', 'DIPEP_CS', 'DIPEP_CT', 'DIPEP_CV', 'DIPEP_CW',
'DIPEP_CY', 'DIPEP_DA', 'DIPEP_DC', 'DIPEP_DD', 'DIPEP_DE', 'DIPEP_DF',
'DIPEP_DG', 'DIPEP_DH', 'DIPEP_DI', 'DIPEP_DK', 'DIPEP_DL', 'DIPEP_DM',
'DIPEP_DN', 'DIPEP_DP', 'DIPEP_DQ', 'DIPEP_DR', 'DIPEP_DS', 'DIPEP_DT',
'DIPEP_DV', 'DIPEP_DW', 'DIPEP_DY', 'DIPEP_EA', 'DIPEP_EC', 'DIPEP_ED',
'DIPEP_EE', 'DIPEP_EF', 'DIPEP_EG', 'DIPEP_EH', 'DIPEP_EI', 'DIPEP_EK',
'DIPEP_EL', 'DIPEP_EM', 'DIPEP_EN', 'DIPEP_EP', 'DIPEP_EQ', 'DIPEP_ER',
'DIPEP_ES', 'DIPEP_ET', 'DIPEP_EV', 'DIPEP_EW', 'DIPEP_EY', 'DIPEP_FA',
```

'DIPEP_FC', 'DIPEP_FD', 'DIPEP_FE', 'DIPEP_FF', 'DIPEP_FG', 'DIPEP_FH',
'DIPEP_FI', 'DIPEP_FK', 'DIPEP_FL', 'DIPEP_FM', 'DIPEP_FN', 'DIPEP_FP',
'DIPEP_FQ', 'DIPEP_FR', 'DIPEP_FS', 'DIPEP_FT', 'DIPEP_FV', 'DIPEP_FW',
'DIPEP_FY', 'DIPEP_GA', 'DIPEP_GC', 'DIPEP_GD', 'DIPEP_GE', 'DIPEP_GF',
'DIPEP_GG', 'DIPEP_GH', 'DIPEP_GI', 'DIPEP_GK', 'DIPEP_GL', 'DIPEP_GM',
'DIPEP_GN', 'DIPEP_GP', 'DIPEP_GQ', 'DIPEP_GR', 'DIPEP_GS', 'DIPEP_GT',
'DIPEP_GV', 'DIPEP_GW', 'DIPEP_GY', 'DIPEP_HA', 'DIPEP_HC', 'DIPEP_HD',
'DIPEP_HE', 'DIPEP_HF', 'DIPEP_HG', 'DIPEP_HH', 'DIPEP_HI', 'DIPEP_HK',
'DIPEP_HL', 'DIPEP_HM', 'DIPEP_HN', 'DIPEP_HP', 'DIPEP_HQ', 'DIPEP_HR',
'DIPEP_HS', 'DIPEP_HT', 'DIPEP_HV', 'DIPEP_HW', 'DIPEP_HY', 'DIPEP_IA',
'DIPEP_IC', 'DIPEP_ID', 'DIPEP_IE', 'DIPEP_IF', 'DIPEP_IG', 'DIPEP_IH',
'DIPEP_II', 'DIPEP_IK', 'DIPEP_IL', 'DIPEP_IM', 'DIPEP_IN', 'DIPEP_IP',
'DIPEP_IQ', 'DIPEP_IR', 'DIPEP_IS', 'DIPEP_IT', 'DIPEP_IV', 'DIPEP_IW',
'DIPEP_IY', 'DIPEP_KA', 'DIPEP_KC', 'DIPEP_KD', 'DIPEP_KE', 'DIPEP_KF',
'DIPEP_KG', 'DIPEP_KH', 'DIPEP_KI', 'DIPEP_KK', 'DIPEP_KL', 'DIPEP_KM',
'DIPEP_KN', 'DIPEP_KP', 'DIPEP_KQ', 'DIPEP_KR', 'DIPEP_KS', 'DIPEP_KT',
'DIPEP_KV', 'DIPEP_KW', 'DIPEP_KY', 'DIPEP_LA', 'DIPEP_LC', 'DIPEP_LD',
'DIPEP_LE', 'DIPEP_LF', 'DIPEP_LG', 'DIPEP_LH', 'DIPEP_LI', 'DIPEP_LK',
'DIPEP_LL', 'DIPEP_LM', 'DIPEP_LN', 'DIPEP_LP', 'DIPEP_LQ', 'DIPEP_LR',
'DIPEP_LS', 'DIPEP_LT', 'DIPEP_LV', 'DIPEP_LW', 'DIPEP_LY', 'DIPEP_MA',
'DIPEP_MC', 'DIPEP_MD', 'DIPEP_ME', 'DIPEP_MF', 'DIPEP_MG', 'DIPEP_MH',
'DIPEP_MI', 'DIPEP_MK', 'DIPEP_ML', 'DIPEP_MM', 'DIPEP_MN', 'DIPEP_MP',
'DIPEP_MQ', 'DIPEP_MR', 'DIPEP_MS', 'DIPEP_MT', 'DIPEP_MV', 'DIPEP_MW',
'DIPEP_MY', 'DIPEP_NA', 'DIPEP_NC', 'DIPEP_ND', 'DIPEP_NE', 'DIPEP_NF',
'DIPEP_NG', 'DIPEP_NH', 'DIPEP_NI', 'DIPEP_NK', 'DIPEP_NL', 'DIPEP_NM',
'DIPEP_NN', 'DIPEP_NP', 'DIPEP_NQ', 'DIPEP_NR', 'DIPEP_NS', 'DIPEP_NT',
'DIPEP_NV', 'DIPEP_NW', 'DIPEP_NY', 'DIPEP_PA', 'DIPEP_PC', 'DIPEP_PD',
'DIPEP_PE', 'DIPEP_PF', 'DIPEP_PG', 'DIPEP_PH', 'DIPEP_PI', 'DIPEP_PK',
'DIPEP_PL', 'DIPEP_PM', 'DIPEP_PN', 'DIPEP_PP', 'DIPEP_PQ', 'DIPEP_PR',
'DIPEP_PS', 'DIPEP_PT', 'DIPEP_PV', 'DIPEP_PW', 'DIPEP_PY', 'DIPEP_QA',
'DIPEP_QC', 'DIPEP_QD', 'DIPEP_QE', 'DIPEP_QF', 'DIPEP_QG', 'DIPEP_QH',
'DIPEP_QI', 'DIPEP_QK', 'DIPEP_QL', 'DIPEP_QM', 'DIPEP_QN', 'DIPEP_QP',
'DIPEP_QQ', 'DIPEP_QR', 'DIPEP_QS', 'DIPEP_QT', 'DIPEP_QV', 'DIPEP_QW',
'DIPEP_QY', 'DIPEP_RA', 'DIPEP_RC', 'DIPEP_RD', 'DIPEP_RE', 'DIPEP_RF',
'DIPEP_RG', 'DIPEP_RH', 'DIPEP_RI', 'DIPEP_RK', 'DIPEP_RL', 'DIPEP_RM',
'DIPEP_RN', 'DIPEP_RP', 'DIPEP_RQ', 'DIPEP_RR', 'DIPEP_RS', 'DIPEP_RT',
'DIPEP_RV', 'DIPEP_RW', 'DIPEP_RY', 'DIPEP_SA', 'DIPEP_SC', 'DIPEP_SD',
'DIPEP_SE', 'DIPEP_SF', 'DIPEP_SG', 'DIPEP_SH', 'DIPEP_SI', 'DIPEP_SK',
'DIPEP_SL', 'DIPEP_SM', 'DIPEP_SN', 'DIPEP_SP', 'DIPEP_SQ', 'DIPEP_SR',
'DIPEP_SS', 'DIPEP_ST', 'DIPEP_SV', 'DIPEP_SW', 'DIPEP_SY', 'DIPEP_TA',
'DIPEP_TC', 'DIPEP_TD', 'DIPEP_TE', 'DIPEP_TF', 'DIPEP_TG', 'DIPEP_TH',
'DIPEP_TI', 'DIPEP_TK', 'DIPEP_TL', 'DIPEP_TM', 'DIPEP_TN', 'DIPEP_TP',
'DIPEP_TQ', 'DIPEP_TR', 'DIPEP_TS', 'DIPEP_TT', 'DIPEP_TV', 'DIPEP_TW',
'DIPEP_TY', 'DIPEP_VA', 'DIPEP_VC', 'DIPEP_VD', 'DIPEP_VE', 'DIPEP_VF',
'DIPEP_VG', 'DIPEP_VH', 'DIPEP_VI', 'DIPEP_VK', 'DIPEP_VL', 'DIPEP_VM',
'DIPEP_VN', 'DIPEP_VP', 'DIPEP_VQ', 'DIPEP_VR', 'DIPEP_VS', 'DIPEP_VT',
'DIPEP_VV', 'DIPEP_VW', 'DIPEP_VY', 'DIPEP_WA', 'DIPEP_WC', 'DIPEP_WD',
'DIPEP_WE', 'DIPEP_WF', 'DIPEP_WG', 'DIPEP_WH', 'DIPEP_WI', 'DIPEP_WK',

```
'DIPEP_WL', 'DIPEP_WM', 'DIPEP_WN', 'DIPEP_WP', 'DIPEP_WQ', 'DIPEP_WR',
'DIPEP_WS', 'DIPEP_WT', 'DIPEP_WV', 'DIPEP_WW', 'DIPEP_WY', 'DIPEP_YA',
'DIPEP_YC', 'DIPEP_YD', 'DIPEP_YE', 'DIPEP_YF', 'DIPEP_YG', 'DIPEP_YH',
'DIPEP_YI', 'DIPEP_YK', 'DIPEP_YL', 'DIPEP_YM', 'DIPEP_YN', 'DIPEP_YP',
'DIPEP_YQ', 'DIPEP_YR', 'DIPEP_YS', 'DIPEP_YT', 'DIPEP_YV', 'DIPEP_YW',
'DIPEP_YY', 'P_LOOP', 'KINASE2', 'GLPL', 'MHD', 'RNBS_A', 'RNBS_B', 'RNBS_C',
'RNBS_D', 'HRD', 'DFG', 'LRR1', 'LRR2', 'NUM_NLR_MOTIFS',
'num_motifs_detectados', 'dist_ploop_kinase2', 'dist_kinase2_glpl',
'dist_glpl_mhd', 'dist_ploop_kinase2_norm', 'dist_kinase2_glpl_norm',
'dist_glpl_mhd_norm', 'has_ploop', 'has_kinase2', 'has_glpl', 'has_mhd']

 Clusters ausentes:
164

Dataset original pós CD-HIT: (8647, 448)


Finalizado em: 12:16:54

Tempo decorrido: 0:00:03.516879


Fim tratamento dos dados…
```

[11]:
```
'''
k-Fold-Cross-Validation

'''

def validacao_cruzada(nome, model, X_train, y_train, groups_train):

    cv = StratifiedGroupKFold(
        n_splits=5, #10
        shuffle=True,
        random_state=42
    )
    mcc_scorer = make_scorer(matthews_corrcoef)
    scoring_dict = {
        'recall': 'recall',
        'roc_auc': 'roc_auc',
        'mcc': mcc_scorer,
        'accuracy': 'accuracy',
        'precision': 'precision',
        'f1': 'f1'
    }
    scores = cross_validate(
        model,
```

```
        X_train,
        y_train,
        groups=groups_train,
        cv=cv,
        scoring=scoring_dict,
        n_jobs=-1
    )
    print(f"Finalizada a validação cruzada para: {nome}")
    return scores
```

[12]:
```
'''
Criação dos modelos/ Treinamento
'''
# para melhor separar os conjuntos de treino e teste considerando os␣
 ↪agrupamentos feitos no CD-HIT para sequências muito parecidas
def split_por_homologia(df):
    print(df.head(30))
    groups = df["cluster"]
    splitter = GroupShuffleSplit(
        n_splits=1,
        test_size=0.2,
        random_state=42
    )
    # cv  = StratifiedGroupKFold(
    #     n_splits = 10,
    #     shuffle=True,
    #     random_state = 42
    # )

    train_idx, test_idx = next(
        splitter.split(df, groups=groups)
        # cv.split(df, df["target"], groups)
    )
    train_df = df.iloc[train_idx]
    test_df = df.iloc[test_idx]
    return train_df, test_df

def calcular_e_salvar_shap(nome, model, X_test, feature_names, output_dir):
    """
    Calcula os SHAP values para modelos de árvore e salva os gráficos␣
 ↪explicativos.
    """
    print(f"\n[SHAP] Inicializando análise de interpretabilidade para: {nome}...
 ↪")

    # 1. Inicializa o explicador otimizado para árvores
```

```
    explainer = shap.TreeExplainer(model)

    # 2. Calcula os SHAP values para o conjunto de teste
    shap_values = explainer.shap_values(X_test)

    # 3. Tratamento de formato (Scikit-Learn Random Forest vs XGBoost)
    # O Random Forest do sklearn retorna uma lista contendo [valores_classe_0,␣
↪valores_classe_1]
    # O XGBoost retorna diretamente a matriz de SHAP values para a classe alvo
    if isinstance(shap_values, list):
        # Selecionamos o índice 1 (classe positiva: proteínas de resistência/
↪alvos)
        shap_values_pos = shap_values[1]
    elif isinstance(shap_values, np.ndarray) and len(shap_values.shape) == 3:
        # Tratamento para versões específicas do SHAP que geram arrays 3D
        shap_values_pos = shap_values[:, :, 1]
    else:
        shap_values_pos = shap_values

    # 4. Gráfico 1: Summary Plot (Beeswarm)
    # Mostra o impacto biológico de cada feature (ex: se alta frequência␣
↪aumenta ou diminui a predição)
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        max_display=20,  # Foco nas top 20 características
        show=False
    )
    # plt.title(f"SHAP Summary Plot (Top 20) - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_summary.png", bbox_inches="tight",␣
↪dpi=300)
    plt.close()

    # 5. Gráfico 2: Bar Plot (Importância Global)
    # Mostra a magnitude média do impacto de cada feature de forma absoluta
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        plot_type="bar",
        max_display=20,
        show=False
    )
```

```
    # plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_bar.png", bbox_inches="tight",␣
↪dpi=300)
    plt.close()

    print(f"[SHAP] Gráficos salvos com sucesso em '{output_dir}/'!")


def calcular_shap_svm(nome, model, feature_names,  X_train, X_test, y_test,␣
↪output_dir):
    # =======================================================
    # IMPORTÂNCIA POR PERMUTAÇÃO PARA O SVM (Novo)
    # =======================================================
    print("\n[SVM] Calculando a importância por permutação dos atributos...")

    # Calcula a permutação no X_test (pode usar X_train se quiser focar no␣
↪ajuste ao treino)
    result_svm = permutation_importance(model, X_test, y_test, n_repeats=5,␣
↪random_state=42, n_jobs=-1)

    # Monta o DataFrame com os resultados
    importancia_svm = pd.DataFrame({
        'feature': columns,
        'importance_mean': result_svm.importances_mean
    }).sort_values('importance_mean', ascending=False)

    print("\nTop 20 features mais importantes para o SVM:")
    print(importancia_svm.head(20))

    # Gera o gráfico de barras igual ao do XGBoost
    top_svm = importancia_svm.head(20)
    plt.figure(figsize=(10,8))
    plt.barh(top_svm["feature"][::-1], top_svm["importance_mean"][::-1],␣
↪color='seagreen')
    plt.xlabel("Queda no Desempenho (Média)")
    plt.ylabel("Feature / Atributo")
    # plt.title("Top 20 Features - SVM Permutation Importance")
    plt.tight_layout()
    plt.savefig(f"{output_dir}/svm_importancia_permutacao.png",␣
↪bbox_inches="tight", dpi=300)
    plt.close()

    idx_back = np.random.choice(
        len(X_train),
        min(100,len(X_train)),
```

```
        replace = False
    )
    back = X_train[idx_back]

    idx_test = np.random.choice(
        len(X_test),
        min(200,len(X_test)),
        replace = False
    )
    x_test_sample = X_test[idx_test]

    print('\nCriando explainer shap...\n')
    explainer = shap.Explainer(model.predict, back)
    shap_values = explainer(x_test_sample, max_evals=1000)
    print("\nShap SHAP:",shap_values.values.shape)

    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values.values,
        x_test_sample,
        feature_names=feature_names,
      #  plot_type="bar",
        max_display=20,
        show=False
    )
    # plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_bar_summary.png",␣
 ↪bbox_inches="tight", dpi=300)
    plt.close()

    print(f"[SVM] Gráfico salvo com sucesso em '{output_dir}/
 ↪svm_importancia_permutacao.png'!")

# ============================================================
# TREINAMENTO - treina os modelos
# ============================================================

def avaliar_modelo(nome, model, X_train, y_train, X_test, y_test):
    print("\n============================================================")
    print(nome)
    print("============================================================")
    model.fit(X_train, y_train)

    probs = model.predict_proba(X_test)[:,1]
    preds = (probs >= 0.4).astype(int)
    print(classification_report(y_test, preds))
```

```
roc = roc_auc_score(y_test, probs)
pr = average_precision_score(y_test, probs)
mcc = matthews_corrcoef(y_test, preds)
f1 = f1_score(y_test, preds)
precision = precision_score(y_test, preds)
recall = recall_score(y_test, preds)
bal_acc = balanced_accuracy_score(y_test, preds)
acc = accuracy_score(y_test, preds)

print("ROC-AUC          :", roc)
print("PR-AUC           :", pr)
print("MCC              :", mcc)
print("F1               :", f1)
print("Precision        :", precision)
print("Recall           :", recall)
print("Balanced Accuracy:", bal_acc)
print("Standard Accuracy:", acc)

# ======================================================
# MATRIZ DE CONFUSÃO
# ======================================================
cm = confusion_matrix(y_test, preds)
plt.figure(figsize=(5,5))
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Negativo", "Positivo"]
)
disp.plot(cmap="Blues")
# plt.title(f"Matriz de Confusão - {nome}")
plt.savefig(
    f"{OUTPUT_DIR}/{nome}_matriz_confusao.png",
    bbox_inches="tight"
)
plt.close()
# ======================================================
# CURVA ROC
# ======================================================
fpr, tpr, _ = roc_curve(y_test, probs)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6,6))
plt.plot(
    fpr,
    tpr,
    label=f"AUC = {roc_auc:.4f}"
)
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
```

```
    plt.ylabel("True Positive Rate")
    # plt.title(f"Curva ROC - {nome}")
    plt.legend(loc="lower right")
    plt.savefig(
        f"{OUTPUT_DIR}/{nome}_roc.png",
        bbox_inches="tight"
    )
    plt.close()

    return {
        "Modelo": nome,
        "ROC_AUC": roc,
        "F1": f1,
        "Precision": precision,
        "Recall": recall,
        "Balanced_Accuracy": bal_acc,
        "Accuracy": acc,
        "MCC":mcc
    }, model


# ========================================================
# Separação Treino/Teste 80/20
# SPLIT POR HOMOLOGIA
# ========================================================
inicio_treinamento = datetime.now()
print("\n\n\nInciando Treinamento dos modelos: ", inicio_treinamento.
 ↪strftime("%H:%M:%S"))

train_df, test_df = split_por_homologia(df)
X_train = train_df[columns]
y_train = train_df["target"]
X_test = test_df[columns]
y_test = test_df["target"]
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ========================================================
# MODELOS
# ========================================================

rf = RandomForestClassifier(
    n_estimators=1000,
    # max_depth = 12,
    max_depth = None,
    min_samples_split=5,
```

```
    max_features=0.2,
    random_state=42,
    class_weight="balanced",
    n_jobs = 1
)

xgb = XGBClassifier(
    n_estimators=1200,
    max_depth=7,
    learning_rate=0.05,
    # early_stopping_rounds=50,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42,
    min_child_weight=3,
    reg_alpha=0.5,
    reg_lambda=2,
    scale_pos_weight=1.0
)


svm = SVC(
    probability=True,
    kernel="rbf",
    class_weight="balanced",
    random_state=42,
    C=2,
    gamma='scale'
)
scores = []
groups_train = train_df["cluster"]

# msc_scorer = make_scorer(matthews_corrcoef)

scoring = {
    'mcc': make_scorer(matthews_corrcoef),
    'recall':  make_scorer(recall_score),
    'precision':  make_scorer(precision_score),
    'f1':  make_scorer(f1_score),
    'roc_auc': 'roc_auc',
}
```

```
Inciando Treinamento dos modelos:  12:16:54
    protein_length     AAC_A     AAC_C     AAC_D     AAC_E     AAC_F  \
```

1	231	0.090909	0.012987	0.069264	0.077922	0.051948
2	704	0.049716	0.007102	0.080966	0.117898	0.041193
7	971	0.045314	0.017508	0.054583	0.071061	0.036045
8	323	0.046440	0.027864	0.052632	0.080495	0.046440
9	929	0.034446	0.034446	0.046286	0.069968	0.053821
10	709	0.053597	0.014104	0.049365	0.071932	0.043724
11	999	0.053053	0.018018	0.070070	0.050050	0.036036
12	1071	0.056956	0.020542	0.059757	0.038282	0.043884
13	1050	0.043810	0.022857	0.053333	0.080952	0.041905
14	904	0.042035	0.022124	0.032080	0.055310	0.056416
15	1152	0.065972	0.015625	0.041667	0.046875	0.030382
16	949	0.033720	0.034773	0.052687	0.067439	0.041096
17	272	0.055147	0.033088	0.040441	0.084559	0.040441
18	428	0.049065	0.023364	0.070093	0.070093	0.051402
19	253	0.059289	0.019763	0.071146	0.051383	0.039526
20	886	0.038375	0.024831	0.060948	0.055305	0.050790
21	910	0.052747	0.017582	0.060440	0.065934	0.052747
22	879	0.035267	0.018203	0.052332	0.073948	0.046644
23	239	0.041841	0.020921	0.083682	0.050209	0.046025
24	126	0.055556	0.031746	0.055556	0.055556	0.039683
25	104	0.028846	0.038462	0.038462	0.067308	0.028846
26	105	0.057143	0.019048	0.076190	0.019048	0.057143
27	716	0.044693	0.023743	0.061453	0.067039	0.046089
28	196	0.061224	0.010204	0.040816	0.091837	0.035714
29	122	0.040984	0.008197	0.081967	0.073770	0.040984
30	1289	0.044220	0.017843	0.061288	0.074476	0.046548
31	846	0.034279	0.017730	0.063830	0.069740	0.047281
32	883	0.055493	0.019253	0.057758	0.070215	0.048698
33	906	0.052980	0.020971	0.055188	0.073951	0.049669
34	343	0.084548	0.011662	0.061224	0.064140	0.037901

	AAC_G	AAC_H	AAC_I	AAC_K	…	dist_ploop_kinase2_norm	\
1	0.077922	0.038961	0.043290	0.082251	…	-1.000000	
2	0.048295	0.009943	0.065341	0.117898	…	-1.000000	
7	0.055613	0.024717	0.057673	0.061792	…	0.094748	
8	0.065015	0.024768	0.061920	0.052632	…	-1.000000	
9	0.051668	0.027987	0.071044	0.059203	…	-1.000000	
10	0.063470	0.021157	0.063470	0.077574	…	-1.000000	
11	0.069069	0.021021	0.043043	0.077077	…	-1.000000	
12	0.090570	0.023343	0.040149	0.059757	…	-1.000000	
13	0.048571	0.024762	0.065714	0.058095	…	-0.023810	
14	0.059735	0.028761	0.065265	0.074115	…	-1.000000	
15	0.110243	0.026910	0.039062	0.046007	…	-1.000000	
16	0.049526	0.025290	0.060063	0.052687	…	0.089568	
17	0.044118	0.029412	0.047794	0.066176	…	-1.000000	
18	0.053738	0.011682	0.072430	0.077103	…	-1.000000	
19	0.063241	0.027668	0.075099	0.102767	…	0.280632	
20	0.053047	0.027088	0.067720	0.082393	…	0.106095	

```
21  0.040659  0.021978  0.071429  0.086813  …                0.100000
22  0.039818  0.036405  0.069397  0.059158  …                0.102389
23  0.033473  0.025105  0.092050  0.079498  …               -1.000000
24  0.055556  0.039683  0.079365  0.095238  …               -1.000000
25  0.067308  0.019231  0.067308  0.076923  …               -1.000000
26  0.057143  0.019048  0.066667  0.076190  …               -1.000000
27  0.051676  0.018156  0.064246  0.071229  …               -1.000000
28  0.035714  0.020408  0.071429  0.076531  …               -1.000000
29  0.032787  0.016393  0.049180  0.073770  …               -1.000000
30  0.038014  0.038014  0.071373  0.074476  …                0.070597
31  0.033097  0.027187  0.075650  0.065012  …                0.106383
32  0.049830  0.022650  0.056625  0.092865  …                0.098528
33  0.045254  0.029801  0.070640  0.071744  …                0.097130
34  0.075802  0.032070  0.075802  0.067055  …               -1.000000

    dist_kinase2_glpl_norm  dist_glpl_mhd_norm  has_ploop  has_kinase2  \
1                -1.000000                   1          0            0
2                -1.000000                   0          0            1
7                 0.108136                   1          1            1
8                -1.000000                   0          0            0
9                -1.000000                   1          0            1
10                0.128350                   0          0            1
11                0.288288                   1          0            1
12                0.202614                   0          0            1
13                0.096190                   1          1            1
14               -1.000000                   0          0            1
15               -1.000000                   0          0            1
16                0.295047                   1          1            1
17                0.312500                   0          0            1
18                0.235981                   1          0            1
19               -1.000000                   1          1            1
20                0.093679                   1          1            1
21               -0.346154                   1          1            1
22                0.145620                   1          1            1
23               -1.000000                   1          0            0
24               -1.000000                   0          0            0
25               -1.000000                   0          0            1
26               -1.000000                   1          0            0
27                0.139665                   1          0            1
28               -1.000000                   0          1            0
29               -1.000000                   0          0            0
30               -0.452289                   1          1            1
31                0.102837                   1          1            1
32                0.412231                   1          1            1
33                0.143488                   1          1            1
34               -1.000000                   0          0            0

    has_glpl  has_mhd  target              id   cluster
```

```
1          0        1       1  NP_001234459.3  POS_1502
2          0        1       1  NP_001308492.1  POS_1005
7          1        4       1    CAQ5402917.1   POS_548
8          0        0       1    CAQ5403257.1  POS_1404
9          0        2       1    CAQ5404663.1   POS_614
10         1        2       1    CAQ5406153.1  POS_1000
11         1        3       1    CAQ5404100.1   POS_504
12         1        2       1    CAQ5404098.1   POS_427
13         1        4       1    CAQ5403806.1   POS_449
14         0        1       1    CAQ5409432.1   POS_681
15         0        1       1    CAQ5408022.1   POS_356
16         1        4       1    CAQ5407207.1   POS_585
17         1        2       1    CAQ5411306.1  POS_1456
18         1        3       1    CAQ5411102.1  POS_1278
19         0        3       1    CAQ5410995.1  POS_1472
20         1        4       1    CAQ5413609.1   POS_723
21         1        4       1    CAQ5413612.1   POS_663
22         1        4       1    CAQ5410853.1   POS_740
23         0        1       1    CAQ5410852.1  POS_1491
24         0        0       1    CAQ5413789.1  POS_1696
25         0        1       1    CAQ5410659.1  POS_1709
26         0        1       1    CAQ5410656.1  POS_1708
27         1        3       1    CAQ5410654.1   POS_994
28         1        2       1    CAQ5410652.1  POS_1573
29         1        1       1    CAQ5410649.1  POS_1699
30         1        4       1    CAQ5410600.1   POS_211
31         1        4       1    CAQ5413906.1   POS_810
32         1        4       1    CAQ5410544.1   POS_729
33         1        4       1    CAQ5413907.1   POS_675
34         0        0       1    CAQ5413931.1  POS_1378

[30 rows x 448 columns]
```

[13]:
```
inicio_otimizacao = datetime.now()
print("\n\n\nInciando Otimização dos modelos: ", inicio_otimizacao.strftime("%H:
  ↪%M:%S"))

cv_t = GroupKFold(n_splits=5)

param_dist_xgb = {
    'n_estimators':[500,800,1200,1500],
    'max_depth':[3,4,5,6,7,10],
    'learning_rate':[0.01,0.03,0.05,0.1],
    'subsample':[0.7,0.8,0.9],
    'colsample_bytree':[0.7,0.8,0.9],
    'min_child_weight':[1,3,5],
    'reg_alpha':[0,0.1,0.5,0.7],
```

```
    'reg_lambda':[1,2,5]
}
# Otimização XGB
print('\n\nOtimizando XGBoost...')
search_xgb = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist_xgb,
    n_iter=50,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_xgb.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o XGB:")
print(search_xgb.best_params_)

print("\nMelhor Recall:")
print(search_xgb.best_score_)

best_params = search_xgb.best_params_
xgb = search_xgb.best_estimator_
```

```
Inciando Otimização dos modelos:  12:16:54


Otimizando XGBoost…
Fitting 5 folds for each of 50 candidates, totalling 250 fits

Melhores parâmetros para o XGB:
{'subsample': 0.7, 'reg_lambda': 2, 'reg_alpha': 0.1, 'n_estimators': 1200,
'min_child_weight': 5, 'max_depth': 5, 'learning_rate': 0.05,
'colsample_bytree': 0.7}

Melhor Recall:
0.8656563971984184
```

[14]:
```
# otimização RF

param_dist_rf = {
    'n_estimators':[500,1000,1200,1500],
    'max_depth':[8,12,16, None],
    'min_samples_split':[2,4,5,8],
    'min_samples_leaf':[1,2,4],
    'max_features':['sqrt','log2',0,3],
    'class_weight':['balanced','balanced_subsample']
}

print('\n\nOtimizando RF...')
search_rf = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist_rf,
    n_iter=40,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_rf.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o RF:")
print(search_rf.best_params_)

print("\nMelhor Recall:")
print(search_rf.best_score_)

rf = search_rf.best_estimator_

rf.set_params(n_jobs=-1)
```

```
Otimizando RF…
Fitting 5 folds for each of 40 candidates, totalling 200 fits

Melhores parâmetros para o RF:
{'n_estimators': 1200, 'min_samples_split': 8, 'min_samples_leaf': 2,
'max_features': 'sqrt', 'max_depth': 8, 'class_weight': 'balanced'}
```

```
Melhor Recall:
0.8250845388265498
```

[14]:
```
RandomForestClassifier(class_weight='balanced', max_depth=8, min_samples_leaf=2,
                       min_samples_split=8, n_estimators=1200, n_jobs=-1,
                       random_state=42)
```

[15]:
```
print('\n\nOtimizando SVM...')
param_dist_svm = {
    'C':[0.5,1,2,5,10,20,50],
    'gamma':[
        'scale',
        0.01,
        0.005,
        0.001,
        0.0005
    ],
    'kernel':['rbf']
}
search_svm = RandomizedSearchCV(
    estimator=svm,
    param_distributions=param_dist_svm,
    n_iter=25,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_svm.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o SVM:")
print(search_svm.best_params_)

print("\nMelhor Recall:")
print(search_svm.best_score_)

svm = search_svm.best_estimator_

fim = datetime.now()
print("\n\nFinalizado otimização dos modelos em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_otimizacao)
```

```
# Fim otimização
```

```
Otimizando SVM…
Fitting 5 folds for each of 25 candidates, totalling 125 fits

Melhores parâmetros para o SVM:
{'kernel': 'rbf', 'gamma': 'scale', 'C': 1}

Melhor Recall:
0.8797550467683726


Finalizado otimização dos modelos em: 12:53:22

Tempo decorrido: 0:36:28.033866
```

[16]:
```
# xgb = search_xgb.best_estimator_

print("\n\n Inciando validação cruzada");
scores_rf = validacao_cruzada('RF',rf,X_train, y_train, groups_train)
scores_xgb = validacao_cruzada('XGB',xgb, X_train, y_train, groups_train)
scores_svm = validacao_cruzada('SVM',svm, X_train, y_train, groups_train)

print("Cross-Validation:RF")
for k,v in scores_rf.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation XGB")
for k,v in scores_xgb.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation SVM")
for k,v in scores_svm.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")


resultados_cv = pd.DataFrame({
    'Modelo': ['RF','XGB','SVM'],
    'Accuracy': [
        scores_rf['test_accuracy'].mean(),
        scores_xgb['test_accuracy'].mean(),
        scores_svm['test_accuracy'].mean(),
```

```
    ],
    'Recall': [
        scores_rf['test_recall'].mean(),
        scores_xgb['test_recall'].mean(),
        scores_svm['test_recall'].mean(),
    ],
    'Precision': [
        scores_rf['test_precision'].mean(),
        scores_xgb['test_precision'].mean(),
        scores_svm['test_precision'].mean(),
    ],
    'F1': [
        scores_rf['test_f1'].mean(),
        scores_xgb['test_f1'].mean(),
        scores_svm['test_f1'].mean(),
    ],
    'ROC_AUC': [
        scores_rf['test_roc_auc'].mean(),
        scores_xgb['test_roc_auc'].mean(),
        scores_svm['test_roc_auc'].mean(),
    ],
    'MCC': [
        scores_rf['test_mcc'].mean(),
        scores_xgb['test_mcc'].mean(),
        scores_svm['test_mcc'].mean(),
    ]
})
print(resultados_cv)
resultados_cv.to_csv(
        f"{OUTPUT_DIR}/metricas_cv.csv",
        index=False
    )
dados = []

for score in scores_rf['test_roc_auc']:
    dados.append(['RF',score])
for score in scores_xgb['test_roc_auc']:
    dados.append(['XGB',score])
for score in scores_svm['test_roc_auc']:
    dados.append(['SVM',score])

grafico = pd.DataFrame(dados, columns=['Modelo','ROC_AUC'])
grafico.boxplot(by='Modelo')
plt.ylabel("ROC-AUC")
# plt.title("10-Fold Cross Validation")
plt.suptitle("")
plt.tight_layout()
```

```
# plt.show()
plt.savefig(
    f"{OUTPUT_DIR}/kfold_cross_validation_roc.png",
    bbox_inches="tight"
)
plt.close()
```

```
 Inciando validação cruzada
Finalizada a validação cruzada para: RF
Finalizada a validação cruzada para: XGB
Finalizada a validação cruzada para: SVM
Cross-Validation:RF
test_recall: 0.7119 ± 0.0126
test_roc_auc: 0.9409 ± 0.0090
test_mcc: 0.8008 ± 0.0109
test_accuracy: 0.9386 ± 0.0031
test_precision: 0.9743 ± 0.0089
test_f1: 0.8226 ± 0.0099
Cross-Validation XGB
test_recall: 0.7690 ± 0.0151
test_roc_auc: 0.9500 ± 0.0094
test_mcc: 0.8320 ± 0.0178
test_accuracy: 0.9480 ± 0.0052
test_precision: 0.9639 ± 0.0168
test_f1: 0.8554 ± 0.0147
Cross-Validation SVM
test_recall: 0.8072 ± 0.0122
test_roc_auc: 0.9546 ± 0.0085
test_mcc: 0.8507 ± 0.0174
test_accuracy: 0.9536 ± 0.0052
test_precision: 0.9540 ± 0.0174
test_f1: 0.8745 ± 0.0139
  Modelo  Accuracy    Recall  Precision        F1   ROC_AUC       MCC
0     RF  0.938558  0.711913   0.974341  0.822647  0.940889  0.800783
1    XGB  0.947955  0.768953   0.963874  0.855402  0.950043  0.832012
2    SVM  0.953594  0.807220   0.954014  0.874481  0.954567  0.850715
```

[17]:
```
################################################
# Treinamento
inicio = datetime.now()
print("\n\n\nInciando treinamento RF: ", inicio.strftime("%H:%M:%S"))
results = []
r1, rf_model = avaliar_modelo(
    "Random Forest",
    rf,
```

```
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r1)
fim = datetime.now()
print("\n\nFinalizado treinamento RF em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("Random Forest", rf_model, X_test, columns,OUTPUT_DIR )

inicio = datetime.now()
print("\n\n\nInciando treinamento XGBoost: ", inicio.strftime("%H:%M:%S"))
r2, xgb_model = avaliar_modelo(
    "XGBoost",
    xgb,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r2)
fim = datetime.now()
print("\n\nFinalizado em XGBoost:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("XGBoost", xgb_model, X_test, columns,OUTPUT_DIR )

inicio = datetime.now()
print("\n\n\nInciando treinamento SVM: ", inicio.strftime("%H:%M:%S"))
r3, svm_model = avaliar_modelo(
    "SVM",
    svm,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r3)
fim = datetime.now()
print("\n\nFinalizado em SVM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_shap_svm("SVM", svm_model,columns, X_train, X_test, y_test, OUTPUT_DIR)

fim = datetime.now()
```

```python
print("\n\nFinalizado fase de treinamento em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_treinamento)

print("\nFim criação/treino dos modelos\n")
```

```
Inciando treinamento RF:  12:54:21

============================================================
Random Forest
============================================================
              precision    recall  f1-score   support

           0       0.94      0.98      0.96      1415
           1       0.90      0.73      0.80       315

    accuracy                           0.94      1730
   macro avg       0.92      0.85      0.88      1730
weighted avg       0.93      0.94      0.93      1730

ROC-AUC          : 0.936021089236637
PR-AUC           : 0.870401583056205
MCC              : 0.7713907183339921
F1               : 0.8035087719298246
Precision        : 0.8980392156862745
Recall           : 0.726984126984127
Balanced Accuracy: 0.85430478434012
Standard Accuracy: 0.9352601156069364


Finalizado treinamento RF em: 12:54:25

Tempo decorrido: 0:00:03.469522

[SHAP] Inicializando análise de interpretabilidade para: Random Forest…
[SHAP] Gráficos salvos com sucesso em 'pipelineAE/'!



Inciando treinamento XGBoost:  12:54:52

============================================================
XGBoost
============================================================
              precision    recall  f1-score   support
```

```
           0       0.95      0.99      0.97      1415
           1       0.96      0.77      0.86       315

    accuracy                           0.95      1730
   macro avg       0.96      0.88      0.91      1730
weighted avg       0.95      0.95      0.95      1730

ROC-AUC          : 0.9448718380167144
PR-AUC           : 0.8982079018650121
MCC              : 0.8368974316915441
F1               : 0.8576449912126538
Precision        : 0.9606299212598425
Recall           : 0.7746031746031746
Balanced Accuracy: 0.8837680183969936
Standard Accuracy: 0.9531791907514451


Finalizado em XGBoost: 12:55:03

Tempo decorrido: 0:00:11.741061

[SHAP] Inicializando análise de interpretabilidade para: XGBoost…
[SHAP] Gráficos salvos com sucesso em 'pipelineAE/'!



Inciando treinamento SVM:  12:55:04

============================================================
SVM
============================================================
              precision    recall  f1-score   support

           0       0.96      0.99      0.97      1415
           1       0.92      0.79      0.85       315

    accuracy                           0.95      1730
   macro avg       0.94      0.89      0.91      1730
weighted avg       0.95      0.95      0.95      1730

ROC-AUC          : 0.9516832127432834
PR-AUC           : 0.9034847607630327
MCC              : 0.826899283377102
F1               : 0.8532423208191127
Precision        : 0.922509225092251
Recall           : 0.7936507936507936
Balanced Accuracy: 0.8894049021257502
Standard Accuracy: 0.9502890173410404
```

```
Finalizado em SVM: 12:55:17

Tempo decorrido: 0:00:12.236279

[SVM] Calculando a importância por permutação dos atributos…

Top 20 features mais importantes para o SVM:
       feature  importance_mean
382   DIPEP_WC         0.001272
299   DIPEP_QW         0.001040
263   DIPEP_PD         0.000809
250   DIPEP_NL         0.000809
265   DIPEP_PF         0.000694
103   DIPEP_FD         0.000694
131   DIPEP_GM         0.000578
216   DIPEP_LS         0.000578
163   DIPEP_ID         0.000578
441  has_ploop         0.000462
422    KINASE2         0.000462
154   DIPEP_HQ         0.000347
289   DIPEP_QK         0.000347
114   DIPEP_FQ         0.000347
398   DIPEP_WV         0.000347
374   DIPEP_VQ         0.000347
102   DIPEP_FC         0.000347
395   DIPEP_WR         0.000347
61    DIPEP_DA         0.000231
111   DIPEP_FM         0.000231

Criando explainer shap…

PermutationExplainer explainer:   4%|         | 8/200 [04:26<2:01:47, 38.06s/it]
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  24.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.0min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  59.6s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.7min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
```

```
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  44.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.7min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  59.0s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  43.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  29.9s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.3min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  27.2s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.2min
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time= 1.5min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=  10.7s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  10.6s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  10.5s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  22.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  27.7s
```

```
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  22.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  12.6s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time= 4.8min
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time= 1.9min
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time= 2.1min
[CV] END …C=1, gamma=scale, kernel=rbf; total time= 2.3min
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time= 4.6min
[CV] END …C=10, gamma=0.001, kernel=rbf; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  36.3s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  46.6s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.7min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  52.5s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 3.5min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  46.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  16.7s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 2.0min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time= 1.1min
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  51.4s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  51.8s
```

```
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 1.1min
[CV] END …C=10, gamma=0.01, kernel=rbf; total time= 4.2min
[CV] END …C=1, gamma=0.001, kernel=rbf; total time= 1.5min
[CV] END …C=50, gamma=0.001, kernel=rbf; total time= 1.1min
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time= 1.9min
[CV] END …C=20, gamma=0.005, kernel=rbf; total time= 4.0min
[CV] END …C=50, gamma=0.005, kernel=rbf; total time= 3.9min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  32.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.5min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.7min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 2.0min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  52.1s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.9min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  23.2s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  38.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  23.6s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  51.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
```

```
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=  11.7s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=  11.5s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  55.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  56.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=  12.3s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=  13.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 1.0min
[CV] END …C=2, gamma=0.001, kernel=rbf; total time= 1.7min
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time= 1.4min
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time= 1.6min
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time= 1.9min
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time= 2.2min
[CV] END …C=20, gamma=0.005, kernel=rbf; total time= 3.6min
[CV] END …C=50, gamma=0.005, kernel=rbf; total time= 3.4min
[CV] END …C=50, gamma=0.01, kernel=rbf; total time= 1.7min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  24.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.3min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  52.1s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.6min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.3min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  42.2s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  21.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 3.4min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  48.3s
```

```
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  48.8s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.2min
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time= 1.5min
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=  11.3s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=  11.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  28.3s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=  10.1s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=  16.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 1.0min
[CV] END …C=10, gamma=0.01, kernel=rbf; total time= 4.8min
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time= 2.0min
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time= 2.1min
[CV] END …C=1, gamma=scale, kernel=rbf; total time= 2.3min
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time= 4.7min
[CV] END …C=50, gamma=0.01, kernel=rbf; total time= 1.7min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  23.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 1.0min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.7min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  46.7s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.6min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 3.4min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
```

```
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  46.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  17.0s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 2.0min
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   9.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=  13.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=  15.6s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=  15.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=  18.4s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=  18.5s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=  10.7s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=  10.8s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  22.1s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=  10.2s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=  13.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  59.5s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time= 1.8min
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time= 1.4min
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time= 1.5min
[CV] END …C=5, gamma=0.01, kernel=rbf; total time= 4.7min
```

```
[CV] END …C=2, gamma=0.01, kernel=rbf; total time= 4.2min
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time= 1.7min
[CV] END …C=10, gamma=0.001, kernel=rbf; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  23.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.8min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  44.1s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.7min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  43.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.3min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  29.2s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.1min
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time= 1.5min
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=  10.7s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=  10.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  28.6s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
```

```
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=  10.2s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=  16.6s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  54.4s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time= 1.1min
[CV] END …C=5, gamma=scale, kernel=rbf; total time= 1.8min
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time= 1.1min
[CV] END …C=1, gamma=0.001, kernel=rbf; total time= 1.5min
[CV] END …C=50, gamma=0.001, kernel=rbf; total time= 1.2min
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time= 1.9min
[CV] END …C=20, gamma=0.005, kernel=rbf; total time= 3.7min
[CV] END …C=50, gamma=0.005, kernel=rbf; total time= 3.5min
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  57.7s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  57.6s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.5min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  43.8s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  21.7s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  21.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  36.0s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  54.2s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
```

```
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  45.1s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  31.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  54.2s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  32.4s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time= 1.0min
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  54.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  54.5s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  50.6s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time= 1.8min
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time= 1.4min
[CV] END …C=2, gamma=0.005, kernel=rbf; total time= 3.9min
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time= 2.2min
[CV] END …C=1, gamma=scale, kernel=rbf; total time= 1.9min
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time= 4.0min
[CV] END …C=50, gamma=scale, kernel=rbf; total time= 1.6min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  56.4s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
```

```
subsample=0.8; total time=  56.5s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  43.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.3min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.9min
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   9.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=  13.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time= 1.0min
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time= 1.0min
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=  16.4s
```

```
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 1.0min
[CV] END …C=10, gamma=0.01, kernel=rbf; total time= 4.8min
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time= 2.0min
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time= 2.4min
[CV] END …C=1, gamma=scale, kernel=rbf; total time= 2.2min
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time= 3.2min
[CV] END …C=50, gamma=scale, kernel=rbf; total time= 2.1min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  32.2s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.5min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  59.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.7min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.9min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  49.3s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.8min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  23.9s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  55.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=  11.7s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=  11.1s
```

```
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  28.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  28.1s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  18.4s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  18.5s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  22.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=  12.1s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=  16.4s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 1.0min
[CV] END …C=10, gamma=0.01, kernel=rbf; total time= 4.7min
[CV] END …C=5, gamma=0.01, kernel=rbf; total time= 4.8min
[CV] END …C=2, gamma=0.01, kernel=rbf; total time= 4.8min
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time= 2.0min
[CV] END …C=50, gamma=0.01, kernel=rbf; total time= 1.3min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  23.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.0min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 1.0min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.8min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  43.2s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
```

```
subsample=0.9; total time= 1.7min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  41.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  53.0s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  44.2s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  26.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  52.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  30.0s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   9.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=  13.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=  15.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time= 1.0min
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=  18.8s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  22.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  27.8s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
```

```
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  22.7s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  12.6s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time= 4.7min
[CV] END …C=5, gamma=0.01, kernel=rbf; total time= 4.8min
[CV] END …C=2, gamma=0.01, kernel=rbf; total time= 4.7min
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time= 2.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  37.6s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  47.1s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 1.0min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.7min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  45.5s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.7min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  59.7s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  42.2s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  51.8s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  44.5s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  48.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  16.8s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.9min
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   9.2s
```

```
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=  13.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=  15.6s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=  15.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=  18.3s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=  18.5s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  10.5s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  10.6s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  10.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=  12.1s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=  10.2s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=  13.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  56.7s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time= 1.8min
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time= 1.5min
[CV] END …C=2, gamma=0.005, kernel=rbf; total time= 3.9min
[CV] END …C=5, gamma=0.005, kernel=rbf; total time= 4.0min
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time= 4.6min
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  59.1s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
```

```
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  55.9s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.3min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  53.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  34.9s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  50.5s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  44.8s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  47.8s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  17.2s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 2.0min
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   9.3s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=  13.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time= 1.0min
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time= 1.0min
[CV] END class_weight=balanced, max_depth=16, max_features=3,
```

```
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=  13.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  59.3s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time= 1.1min
[CV] END …C=5, gamma=scale, kernel=rbf; total time= 1.8min
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time= 1.1min
[CV] END …C=1, gamma=0.001, kernel=rbf; total time= 1.5min
[CV] END …C=50, gamma=0.001, kernel=rbf; total time= 1.1min
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time= 2.0min
[CV] END …C=20, gamma=0.005, kernel=rbf; total time= 3.9min
[CV] END …C=50, gamma=0.005, kernel=rbf; total time= 4.0min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  31.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.5min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  59.1s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.7min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.9min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  50.7s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.8min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  23.7s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  52.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
```

```
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=  11.7s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=  11.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  28.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  57.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  28.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=  12.1s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=  13.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  59.9s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time= 1.5min
[CV] END …C=5, gamma=scale, kernel=rbf; total time= 2.2min
[CV] END …C=2, gamma=0.005, kernel=rbf; total time= 3.9min
[CV] END …C=5, gamma=0.005, kernel=rbf; total time= 3.8min
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time= 3.2min
[CV] END …C=50, gamma=scale, kernel=rbf; total time= 2.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  36.8s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  46.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.8min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  55.2s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  58.7s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  42.9s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  31.7s
```

```
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.5min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.2min
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time= 1.5min
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=  11.5s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=  11.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  28.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  28.3s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  22.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  12.7s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time= 4.7min
[CV] END …C=5, gamma=0.01, kernel=rbf; total time= 4.8min
[CV] END …C=2, gamma=0.01, kernel=rbf; total time= 4.8min
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time= 2.0min
[CV] END …C=50, gamma=0.01, kernel=rbf; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
```

```
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  54.0s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  53.8s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  36.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  54.8s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  44.8s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  29.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  54.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  27.9s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time= 1.0min
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  54.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  54.6s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  52.4s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time= 1.1min
[CV] END …C=5, gamma=scale, kernel=rbf; total time= 2.2min
[CV] END …C=2, gamma=0.005, kernel=rbf; total time= 3.9min
[CV] END …C=5, gamma=0.005, kernel=rbf; total time= 4.1min
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time= 4.6min
[CV] END …C=50, gamma=0.01, kernel=rbf; total time= 1.7min
```

```
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  38.5s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  45.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.7min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  42.9s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  21.7s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  33.0s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  29.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.3min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  28.6s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.2min
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time= 1.5min
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  18.6s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=  11.4s
```

```
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  21.8s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=  10.1s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=  16.4s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  53.9s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time= 1.4min
[CV] END …C=5, gamma=scale, kernel=rbf; total time= 2.2min
[CV] END …C=2, gamma=0.005, kernel=rbf; total time= 3.9min
[CV] END …C=5, gamma=0.005, kernel=rbf; total time= 4.0min
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time= 3.2min
[CV] END …C=50, gamma=scale, kernel=rbf; total time= 2.0min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  37.4s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  47.0s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.7min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  55.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 3.6min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  27.1s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  19.4s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  58.8s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
```

```
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time= 1.1min
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  50.9s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  51.4s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 1.1min
[CV] END …C=10, gamma=0.01, kernel=rbf; total time= 4.0min
[CV] END …C=1, gamma=0.001, kernel=rbf; total time= 1.5min
[CV] END …C=50, gamma=0.001, kernel=rbf; total time= 1.1min
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time= 2.0min
[CV] END …C=20, gamma=0.005, kernel=rbf; total time= 4.0min
[CV] END …C=50, gamma=0.005, kernel=rbf; total time= 3.8min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  23.6s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.7min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  42.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  21.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
```

```
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  40.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  29.9s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.6min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  50.7s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  27.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  54.5s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  31.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=  11.6s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=  11.4s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  55.9s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  56.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=  12.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  27.4s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  22.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  12.5s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time= 4.8min
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time= 1.9min
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time= 2.4min
[CV] END …C=1, gamma=scale, kernel=rbf; total time= 2.3min
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time= 3.2min
[CV] END …C=50, gamma=scale, kernel=rbf; total time= 2.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
```

```
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.3min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  55.0s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  54.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  31.8s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  29.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.3min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  28.4s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  53.2s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  30.1s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time= 1.0min
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  54.1s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  51.7s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  51.3s
```

```
[CV] END …C=2, gamma=0.001, kernel=rbf; total time= 1.7min
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time= 1.4min
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time= 1.5min
[CV] END …C=1, gamma=0.001, kernel=rbf; total time= 1.9min
[CV] END …C=50, gamma=0.001, kernel=rbf; total time= 1.6min
[CV] END …C=5, gamma=0.005, kernel=rbf; total time= 4.1min
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time= 3.3min
[CV] END …C=10, gamma=0.001, kernel=rbf; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  32.5s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.4min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  59.1s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.7min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.9min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  50.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 4.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  24.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  58.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=  11.5s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=  11.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  28.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  28.2s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  18.4s
```

```
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  18.5s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  28.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  28.1s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  22.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  12.7s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time= 4.7min
[CV] END …C=5, gamma=0.01, kernel=rbf; total time= 4.9min
[CV] END …C=2, gamma=0.01, kernel=rbf; total time= 4.8min
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time= 2.0min

PermutationExplainer explainer: 201it [2:02:43, 36.82s/it]


Shap SHAP: (200, 445)
[SVM] Gráfico salvo com sucesso em 'pipelineAE/svm_importancia_permutacao.png'!


Finalizado fase de treinamento em: 15:01:25

Tempo decorrido: 2:44:30.943389

Fim criação/treino dos modelos


<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>
```

[18]:
```
'''
Avaliação e Validação dos modelos
'''
print("\n\nInciando Validação dos modelos\n")
# verificar importância dos atributos/feature para o Random Forest
importancia = pd.DataFrame({"feature":columns,"importance":rf_model.
 ↪feature_importances_})
importancia = importancia.sort_values("importance",ascending = False)
print("\nTop 20 features: Random Forest")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh( top["feature"][::-1], top["importance"][::-1] )
plt.xlabel("Importância")
```

```
plt.ylabel("Feature")
# plt.title("Top 20 Features - Random Forest")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/rf_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

# verificar importância dos atributos/feature para o XGBoost
importancia = pd.DataFrame({"feature":columns, "importance":xgb_model.
 ↪feature_importances_})
importancia = importancia.sort_values("importance", ascending = False)
print("\nTop 20 features: XGBoost")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh(top["feature"][::-1], top["importance"][::-1])
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - XGBoost")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/xgb_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

print("\nCOMPARAÇÃO MODELOS")
print(pd.DataFrame(results))

results_df = pd.DataFrame(results)
results_df.to_csv(
        f"{OUTPUT_DIR}/metricas_comparacao.csv",
        index=False
    )

metrics = [
    "ROC_AUC",
    "F1",
    "Precision",
    "Recall",
    "Balanced_Accuracy"
]

for metric in metrics:
```

```
    plt.figure(figsize=(8,5))
    plt.bar(
        results_df["Modelo"],
        results_df[metric]
    )

    plt.ylim(0, 1)
    plt.ylabel(metric)
    # plt.title(f"Comparação dos Modelos - {metric}")
    for i, v in enumerate(results_df[metric]):
        plt.text(i, v + 0.01, f"{v:.3f}", ha='center')

    plt.savefig(
        f"{OUTPUT_DIR}/comparacao_{metric}.png",
        bbox_inches="tight"
    )
    plt.close()

modelos = {
    "Random Forest":rf_model,
    "XGBoost":xgb_model,
    "SVM":svm_model
    }
plt.figure(figsize=(7,7))
for nome, model in modelos.items():
    probs = model.predict_proba(X_test)[:,1]
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.plot(
        fpr,
        tpr,
        label=f"{nome} AUC={roc_auc:.3f}"
    )
plt.plot([0,1],[0,1],'--')
plt.xlabel("FPR")
plt.ylabel("TPR")
# plt.title("Comparação ROC")
plt.legend()
plt.savefig(f"{OUTPUT_DIR}/roc_comparativa.png")
plt.close()

print("Fim avaliação / validação modelos ")
```

```
Inciando Validação dos modelos
```

```
Top 20 features: Random Forest
                     feature  importance
10                     AAC_L    0.048118
444                  has_mhd    0.042918
433           NUM_NLR_MOTIFS    0.042768
422                  KINASE2    0.032426
1                      AAC_A    0.032199
204                DIPEP_LE    0.029217
440       dist_glpl_mhd_norm    0.027720
209                DIPEP_LK    0.027437
441               has_ploop    0.024398
250                DIPEP_NL    0.022025
421                  P_LOOP    0.020472
439   dist_kinase2_glpl_norm    0.019760
0             protein_length    0.017984
438  dist_ploop_kinase2_norm    0.017369
13                     AAC_P    0.016360
437            dist_glpl_mhd    0.015636
215                DIPEP_LR    0.015282
190                DIPEP_KL    0.014952
150                DIPEP_HL    0.014606
434    num_motifs_detectados    0.013959

Top 20 features: XGBoost
                   feature  importance
444                has_mhd    0.106355
422                KINASE2    0.090268
433         NUM_NLR_MOTIFS    0.088651
441              has_ploop    0.029028
1                    AAC_A    0.013887
10                   AAC_L    0.013609
434  num_motifs_detectados    0.011607
428                RNBS_D    0.010637
6                    AAC_G    0.010580
4                    AAC_E    0.008425
421                P_LOOP    0.007436
440     dist_glpl_mhd_norm    0.007020
204               DIPEP_LE    0.005756
13                   AAC_P    0.005040
146               DIPEP_HG    0.004948
393               DIPEP_WP    0.004888
382               DIPEP_WC    0.004753
256               DIPEP_NS    0.004333
385               DIPEP_WF    0.004202
0           protein_length    0.003982

COMPARAÇÃO MODELOS
          Modelo   ROC_AUC        F1  Precision    Recall  Balanced_Accuracy  \
```

```
0  Random Forest  0.936021  0.803509   0.898039  0.726984          0.854305
1        XGBoost  0.944872  0.857645   0.960630  0.774603          0.883768
2            SVM  0.951683  0.853242   0.922509  0.793651          0.889405

   Accuracy       MCC
0  0.935260  0.771391
1  0.953179  0.836897
2  0.950289  0.826899
Fim avaliação / validação modelos
```

[19]:

```
'''
Funções para rodar a aplicação
'''

df_validacao = []

def validar_motifs(df,planta,modelo):
    print("\nVALIDAÇÃO BIOLÓGICA\n")
    counts = {}
    for motif_name, pattern in motifs.items():
        counts[motif_name] = 0
        for seq in df["protein_seq"]:
            if re.search(pattern, seq):
                counts[motif_name] += 1
    for k, v in counts.items():
        print(k, ":", v)

    counts['planta'] = planta
    counts['modelo'] = modelo

    df_validacao.append(counts)
    df_counts = pd.DataFrame([counts])
    df_counts.to_csv(
        f"{OUTPUT_DIR}/validacao_biologica_{modelo}_{planta}.csv",
        index=False
    )



def rodar_transdecoder(fasta_nt):
    cmd1 = [
        "TransDecoder.LongOrfs",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
```

```
    with open(fasta_nt+"_output1.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd1,
                       stdout=log_file,
                       stderr=subprocess.STDOUT,
                       text=True)

    cmd2 = [
        "TransDecoder.Predict",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output2.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd2,
                       stdout=log_file,
                       stderr=subprocess.STDOUT,
                       text=True)
    pep_file = fasta_nt + ".transdecoder.pep"
    return pep_file


def predizer_transcritos(fasta_nt,model,nmodel, output, planta):
    probabilidade = 0.85
    candidatos = []
    pep_file = rodar_transdecoder(fasta_nt)
    for record in SeqIO.parse(fasta_nt+".transdecoder.pep", "fasta"):
        protein = str(record.seq)
        feats = extrair_features_proteina(protein)
        if feats is None:
            continue
        X = scaler.transform([feats])
        prob = model.predict_proba(X)[0][1]
        candidatos.append({
            "id": record.id,
            "descricao": record.description,
            "protein_length": len(protein),
            "probabilidade_resistencia": prob,
            "protein_seq": protein
        })

    candidatos_df = pd.DataFrame(candidatos)
    candidatos_df = candidatos_df.sort_values(
        "probabilidade_resistencia",
        ascending=False
    )
```

```
    # top = candidatos_df.head(40)
    top = candidatos_df[candidatos_df["probabilidade_resistencia"] >=␣
 ↪probabilidade]
    print("\nTOP CANDIDATOS com probabilidade de >= ")
    print(probabilidade)
    print(fasta_nt+", MODELO: "+nmodel)
    print(top[[
        "id",
        "descricao",
        "protein_length",
        "probabilidade_resistencia"
    ]])

    validar_motifs(top,planta,nmodel)

    candidatos_df.to_csv(
        f"{OUTPUT_DIR}/{output}",
        index=False
    )
```

[20]:
```
'''
Aplicação - Modelo XGBoost
'''
print("\n\n=================Inciando Aplicação: Predição de␣
 ↪Genes===============\n\n")
print("\n\n Predição Modelo XGBoost")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))

print("\nPredição para ARABIDOPSIS\n")
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_arabidopsis_xgboost.
 ↪csv",planta="Arabidopsis thaliana")

fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
=================Inciando Aplicação: Predição de Genes===============


 Predição Modelo XGBoost
```

```
Inciando predição ARABIDOPSIS:  15:01:26

Predição para ARABIDOPSIS


TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/arabidopsis_transcriptoma.fasta, MODELO: XGBoost
                      id                                                descricao  \
4074   NM_001344104.1.p1  NM_001344104.1.p1 GENE.NM_001344104.1~~NM_0013…
8895      NM_122936.5.p1  NM_122936.5.p1 GENE.NM_122936.5~~NM_122936.5.p…
4358   NM_001344485.1.p1  NM_001344485.1.p1 GENE.NM_001344485.1~~NM_0013…
4359   NM_001344486.1.p1  NM_001344486.1.p1 GENE.NM_001344486.1~~NM_0013…
1789   NM_001341147.1.p1  NM_001341147.1.p1 GENE.NM_001341147.1~~NM_0013…
…                   …                                                   …
5555      NM_115015.3.p1  NM_115015.3.p1 GENE.NM_115015.3~~NM_115015.3.p…
10589     NM_180802.2.p1  NM_180802.2.p1 GENE.NM_180802.2~~NM_180802.2.p…
4533   NM_001344683.1.p1  NM_001344683.1.p1 GENE.NM_001344683.1~~NM_0013…
4536   NM_001344685.1.p1  NM_001344685.1.p1 GENE.NM_001344685.1~~NM_0013…
225    NM_001085250.2.p1  NM_001085250.2.p1 GENE.NM_001085250.2~~NM_0010…

       protein_length  probabilidade_resistencia
4074              902                   0.999910
8895              902                   0.999910
4358              909                   0.999846
4359              909                   0.999846
1789             1362                   0.999841
…                …                            …
5555             1254                   0.876597
10589            1373                   0.868339
4533              544                   0.858416
4536              544                   0.858416
225              1211                   0.855503

[178 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 170
KINASE2 : 27
GLPL : 159
MHD : 143
RNBS_A : 0
RNBS_B : 166
RNBS_C : 176
RNBS_D : 176
```

```
HRD : 6
DFG : 45
LRR1 : 39
LRR2 : 95


Finalizado predição ARABIDOPSIS: 15:03:31

Tempo decorrido: 0:02:05.235225
```

[21]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_melongena_xgboost.csv",planta="Solanum␣
 ↪melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para MELONGENA



Inciando predição MELONGENA:  15:03:31

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/melongena_transcriptoma.fasta, MODELO: XGBoost
                      id                                                descricao  \
274   GBEF01001042.1.p1  GBEF01001042.1.p1 GENE.GBEF01001042.1~~GBEF010…
4488  GBEF01026076.1.p1  GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010…
3221  GBEF01023314.1.p1  GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010…
3220  GBEF01023313.1.p1  GBEF01023313.1.p1 GENE.GBEF01023313.1~~GBEF010…
4338  GBEF01025818.1.p1  GBEF01025818.1.p1 GENE.GBEF01025818.1~~GBEF010…
4337  GBEF01025817.1.p1  GBEF01025817.1.p1 GENE.GBEF01025817.1~~GBEF010…
4489  GBEF01026077.1.p1  GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010…
9160  GBEF01044249.1.p1  GBEF01044249.1.p1 GENE.GBEF01044249.1~~GBEF010…
9238  GBEF01044458.1.p1  GBEF01044458.1.p1 GENE.GBEF01044458.1~~GBEF010…
9159  GBEF01044248.1.p1  GBEF01044248.1.p1 GENE.GBEF01044248.1~~GBEF010…
9162  GBEF01044251.1.p1  GBEF01044251.1.p1 GENE.GBEF01044251.1~~GBEF010…
9158  GBEF01044247.1.p1  GBEF01044247.1.p1 GENE.GBEF01044247.1~~GBEF010…
789   GBEF01005143.1.p1  GBEF01005143.1.p1 GENE.GBEF01005143.1~~GBEF010…
9161  GBEF01044250.1.p1  GBEF01044250.1.p1 GENE.GBEF01044250.1~~GBEF010…
3223  GBEF01023316.1.p1  GBEF01023316.1.p1 GENE.GBEF01023316.1~~GBEF010…
9163  GBEF01044252.1.p1  GBEF01044252.1.p1 GENE.GBEF01044252.1~~GBEF010…
```

3222	GBEF01023315.1.p1	GBEF01023315.1.p1	GENE.GBEF01023315.1~~GBEF010…
1845	GBEF01016702.1.p1	GBEF01016702.1.p1	GENE.GBEF01016702.1~~GBEF010…
6005	GBEF01030217.1.p1	GBEF01030217.1.p1	GENE.GBEF01030217.1~~GBEF010…
263	GBEF01000989.1.p1	GBEF01000989.1.p1	GENE.GBEF01000989.1~~GBEF010…
262	GBEF01000988.1.p1	GBEF01000988.1.p1	GENE.GBEF01000988.1~~GBEF010…
261	GBEF01000987.1.p1	GBEF01000987.1.p1	GENE.GBEF01000987.1~~GBEF010…
264	GBEF01000990.1.p1	GBEF01000990.1.p1	GENE.GBEF01000990.1~~GBEF010…
9236	GBEF01044457.1.p2	GBEF01044457.1.p2	GENE.GBEF01044457.1~~GBEF010…
9240	GBEF01044459.1.p1	GBEF01044459.1.p1	GENE.GBEF01044459.1~~GBEF010…
7149	GBEF01034741.1.p1	GBEF01034741.1.p1	GENE.GBEF01034741.1~~GBEF010…
4586	GBEF01026239.1.p2	GBEF01026239.1.p2	GENE.GBEF01026239.1~~GBEF010…
2372	GBEF01017801.1.p1	GBEF01017801.1.p1	GENE.GBEF01017801.1~~GBEF010…
7642	GBEF01040892.1.p1	GBEF01040892.1.p1	GENE.GBEF01040892.1~~GBEF010…
7023	GBEF01034428.1.p1	GBEF01034428.1.p1	GENE.GBEF01034428.1~~GBEF010…
4776	GBEF01026722.1.p1	GBEF01026722.1.p1	GENE.GBEF01026722.1~~GBEF010…

	protein_length	probabilidade_resistencia
274	867	0.999390
4488	892	0.999236
3221	872	0.998824
3220	881	0.998343
4338	886	0.998331
4337	886	0.998331
4489	866	0.998241
9160	579	0.994658
9238	851	0.994178
9159	706	0.993730
9162	532	0.992857
9158	713	0.991881
789	504	0.989200
9161	572	0.988719
3223	874	0.986296
9163	525	0.977429
3222	883	0.965345
1845	914	0.959751
6005	532	0.951578
263	456	0.943455
262	456	0.943455
261	456	0.943455
264	456	0.943455
9236	244	0.928313
9240	244	0.928313
7149	417	0.924122
4586	408	0.904949
2372	611	0.889311
7642	703	0.884567
7023	1126	0.868796
4776	378	0.850839

```
VALIDAÇÃO BIOLÓGICA

P_LOOP : 15
KINASE2 : 12
GLPL : 24
MHD : 16
RNBS_A : 0
RNBS_B : 14
RNBS_C : 24
RNBS_D : 31
HRD : 1
DFG : 1
LRR1 : 4
LRR2 : 14


Finalizado predição MELONGENA: 15:05:22

Tempo decorrido: 0:01:50.569010
```

[22]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_tuberosum_xgboost.csv",planta="Solanum␣
 ↪tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para TUBEROSUM




Inciando predição TUBEROSUM:  15:05:22

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/tuberosum_transcriptoma.fasta, MODELO: XGBoost
                       id                                                 descricao  \
7736   XM_015304639.1.p1  XM_015304639.1.p1 GENE.XM_015304639.1~~XM_0153…
7739   XM_015304642.1.p1  XM_015304642.1.p1 GENE.XM_015304642.1~~XM_0153…
7737   XM_015304640.1.p1  XM_015304640.1.p1 GENE.XM_015304640.1~~XM_0153…
9242   XM_015312842.1.p1  XM_015312842.1.p1 GENE.XM_015312842.1~~XM_0153…
2842   XM_006358335.2.p1  XM_006358335.2.p1 GENE.XM_006358335.2~~XM_0063…
```

```
…                    …                                                    …
10194  XM_015314540.1.p1  XM_015314540.1.p1 GENE.XM_015314540.1~~XM_0153…
10197  XM_015314541.1.p1  XM_015314541.1.p1 GENE.XM_015314541.1~~XM_0153…
10198  XM_015314542.1.p1  XM_015314542.1.p1 GENE.XM_015314542.1~~XM_0153…
10190  XM_015314538.1.p1  XM_015314538.1.p1 GENE.XM_015314538.1~~XM_0153…
10192  XM_015314539.1.p1  XM_015314539.1.p1 GENE.XM_015314539.1~~XM_0153…

       protein_length  probabilidade_resistencia
7736             1023                   0.999985
7739             1267                   0.999975
7737             1271                   0.999970
9242             1216                   0.999967
2842              978                   0.999964
…                 …                      …
10194             365                   0.855786
10197             365                   0.855786
10198             365                   0.855786
10190             365                   0.855786
10192             365                   0.855786

[342 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 280
KINASE2 : 170
GLPL : 286
MHD : 299
RNBS_A : 0
RNBS_B : 255
RNBS_C : 311
RNBS_D : 331
HRD : 3
DFG : 55
LRR1 : 83
LRR2 : 147


Finalizado predição TUBEROSUM: 15:07:31

Tempo decorrido: 0:02:08.987252
```

[23]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
```

```
predizer_transcritos(transcriptoma_physalis_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_physalis_xgboost.csv",planta="Physalis␣
 ↪peruviana")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_xgboost.csv")
```

```
Predição para PHYSALIS




Inciando predição PHYSALIS:  15:07:31

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/physalis_transcriptoma.fasta, MODELO: XGBoost
                      id                                                 descricao  \
10249  IABH01023929.1.p1  IABH01023929.1.p1 GENE.IABH01023929.1~~IABH010…
10240  IABH01023909.1.p1  IABH01023909.1.p1 GENE.IABH01023909.1~~IABH010…
711    IABH01002505.1.p1  IABH01002505.1.p1 GENE.IABH01002505.1~~IABH010…
710    IABH01002504.1.p1  IABH01002504.1.p1 GENE.IABH01002504.1~~IABH010…
10242  IABH01023913.1.p1  IABH01023913.1.p1 GENE.IABH01023913.1~~IABH010…
…                    …                                                  …
9414   IABH01022562.1.p4  IABH01022562.1.p4 GENE.IABH01022562.1~~IABH010…
9429   IABH01022568.1.p3  IABH01022568.1.p3 GENE.IABH01022568.1~~IABH010…
9422   IABH01022566.1.p4  IABH01022566.1.p4 GENE.IABH01022566.1~~IABH010…
9404   IABH01022555.1.p4  IABH01022555.1.p4 GENE.IABH01022555.1~~IABH010…
5042   IABH01015037.1.p1  IABH01015037.1.p1 GENE.IABH01015037.1~~IABH010…

       protein_length  probabilidade_resistencia
10249             530                   0.999965
10240             849                   0.999946
711               889                   0.999858
710               889                   0.999858
10242             852                   0.999825
…                 …                      …
9414              209                   0.860212
9429              209                   0.860212
9422              209                   0.860212
9404              209                   0.860212
5042              627                   0.855169

[78 rows x 4 columns]
```

```
VALIDAÇÃO BIOLÓGICA

P_LOOP : 51
KINASE2 : 30
GLPL : 59
MHD : 65
RNBS_A : 0
RNBS_B : 60
RNBS_C : 74
RNBS_D : 78
HRD : 1
DFG : 5
LRR1 : 7
LRR2 : 31


Finalizado predição TUBEROSUM: 15:09:35

Tempo decorrido: 0:02:04.310116

Arquivo salvo:
pipelineAE/predicoes_xgboost.csv
```

[24]:
```
print(df_validacao)
```

```
[{'P_LOOP': 170, 'KINASE2': 27, 'GLPL': 159, 'MHD': 143, 'RNBS_A': 0, 'RNBS_B':
166, 'RNBS_C': 176, 'RNBS_D': 176, 'HRD': 6, 'DFG': 45, 'LRR1': 39, 'LRR2': 95,
'planta': 'Arabidopsis thaliana', 'modelo': 'XGBoost'}, {'P_LOOP': 15,
'KINASE2': 12, 'GLPL': 24, 'MHD': 16, 'RNBS_A': 0, 'RNBS_B': 14, 'RNBS_C': 24,
'RNBS_D': 31, 'HRD': 1, 'DFG': 1, 'LRR1': 4, 'LRR2': 14, 'planta': 'Solanum
melongena', 'modelo': 'XGBoost'}, {'P_LOOP': 280, 'KINASE2': 170, 'GLPL': 286,
'MHD': 299, 'RNBS_A': 0, 'RNBS_B': 255, 'RNBS_C': 311, 'RNBS_D': 331, 'HRD': 3,
'DFG': 55, 'LRR1': 83, 'LRR2': 147, 'planta': 'Solanum tuberosum', 'modelo':
'XGBoost'}, {'P_LOOP': 51, 'KINASE2': 30, 'GLPL': 59, 'MHD': 65, 'RNBS_A': 0,
'RNBS_B': 60, 'RNBS_C': 74, 'RNBS_D': 78, 'HRD': 1, 'DFG': 5, 'LRR1': 7, 'LRR2':
31, 'planta': 'Physalis peruviana', 'modelo': 'XGBoost'}]
```

[25]:
```
'''
Aplicação - Modelo Random Forest
'''
print("\n\n Predição Modelo RF")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪rf_model,"RF",output="predicoes_arabidopsis_rf.csv",planta="Arabidopsis␣
 ↪thaliana")
fim = datetime.now()
```

```
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
 Predição Modelo RF

Predição para ARABIDOPSIS



Inciando predição ARABIDOPSIS:  15:09:35

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/arabidopsis_transcriptoma.fasta, MODELO: RF
                     id                                                descricao  \
9182     NM_123739.3.p1  NM_123739.3.p1 GENE.NM_123739.3~~NM_123739.3.p…
4376  NM_001344504.1.p1  NM_001344504.1.p1 GENE.NM_001344504.1~~NM_0013…
4375  NM_001344503.1.p1  NM_001344503.1.p1 GENE.NM_001344503.1~~NM_0013…
4374  NM_001344502.1.p1  NM_001344502.1.p1 GENE.NM_001344502.1~~NM_0013…
4373  NM_001344501.1.p1  NM_001344501.1.p1 GENE.NM_001344501.1~~NM_0013…
…                   …                                                 …
4265  NM_001344370.1.p1  NM_001344370.1.p1 GENE.NM_001344370.1~~NM_0013…
4269  NM_001344373.1.p1  NM_001344373.1.p1 GENE.NM_001344373.1~~NM_0013…
4260  NM_001344367.1.p1  NM_001344367.1.p1 GENE.NM_001344367.1~~NM_0013…
9312     NM_124015.2.p1  NM_124015.2.p1 GENE.NM_124015.2~~NM_124015.2.p…
4458  NM_001344597.1.p1  NM_001344597.1.p1 GENE.NM_001344597.1~~NM_0013…

      protein_length  probabilidade_resistencia
9182             849                   0.971832
4376             849                   0.971832
4375             849                   0.971832
4374             849                   0.971832
4373             849                   0.971832
…                …                          …
4265             921                   0.853062
4269             921                   0.853062
4260             921                   0.853062
9312            1128                   0.850640
4458            1001                   0.850376

[128 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 128
```

```
KINASE2 : 26
GLPL : 126
MHD : 116
RNBS_A : 0
RNBS_B : 122
RNBS_C : 127
RNBS_D : 128
HRD : 6
DFG : 30
LRR1 : 29
LRR2 : 77


Finalizado predição ARABIDOPSIS: 15:25:41

Tempo decorrido: 0:16:06.196619
```

[26]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
 ↪rf_model,'RF',output="predicoes_melongena_rf.csv",planta="Solanum melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para MELONGENA



Inciando predição MELONGENA:  15:25:42

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/melongena_transcriptoma.fasta, MODELO: RF
                      id                                                 descricao  \
3221  GBEF01023314.1.p1  GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010…
3220  GBEF01023313.1.p1  GBEF01023313.1.p1 GENE.GBEF01023313.1~~GBEF010…
274   GBEF01001042.1.p1  GBEF01001042.1.p1 GENE.GBEF01001042.1~~GBEF010…
9238  GBEF01044458.1.p1  GBEF01044458.1.p1 GENE.GBEF01044458.1~~GBEF010…
4489  GBEF01026077.1.p1  GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010…
4488  GBEF01026076.1.p1  GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010…
4338  GBEF01025818.1.p1  GBEF01025818.1.p1 GENE.GBEF01025818.1~~GBEF010…
4337  GBEF01025817.1.p1  GBEF01025817.1.p1 GENE.GBEF01025817.1~~GBEF010…
3222  GBEF01023315.1.p1  GBEF01023315.1.p1 GENE.GBEF01023315.1~~GBEF010…
3223  GBEF01023316.1.p1  GBEF01023316.1.p1 GENE.GBEF01023316.1~~GBEF010…
```

```
      protein_length  probabilidade_resistencia
3221             872                   0.965632
3220             881                   0.961989
274              867                   0.940646
9238             851                   0.928635
4489             866                   0.926315
4488             892                   0.924289
4338             886                   0.905055
4337             886                   0.905055
3222             883                   0.889076
3223             874                   0.885301

VALIDAÇÃO BIOLÓGICA

P_LOOP : 10
KINASE2 : 10
GLPL : 10
MHD : 7
RNBS_A : 0
RNBS_B : 3
RNBS_C : 6
RNBS_D : 10
HRD : 0
DFG : 0
LRR1 : 4
LRR2 : 7


Finalizado predição MELONGENA: 15:39:54

Tempo decorrido: 0:14:12.895110
```

[27]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
  ↪rf_model,"RF",output="predicoes_tuberosum_rf.csv",planta="Solanum tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para TUBEROSUM
```

```
Inciando predição TUBEROSUM:  15:39:54

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/tuberosum_transcriptoma.fasta, MODELO: RF
                      id                                                descricao  \
7738   XM_015304641.1.p1  XM_015304641.1.p1 GENE.XM_015304641.1~~XM_0153…
7737   XM_015304640.1.p1  XM_015304640.1.p1 GENE.XM_015304640.1~~XM_0153…
1986   XM_006356384.2.p1  XM_006356384.2.p1 GENE.XM_006356384.2~~XM_0063…
8115   XM_015305504.1.p1  XM_015305504.1.p1 GENE.XM_015305504.1~~XM_0153…
8116   XM_015305505.1.p1  XM_015305505.1.p1 GENE.XM_015305505.1~~XM_0153…
…                    …                                                 …
9816   XM_015313889.1.p1  XM_015313889.1.p1 GENE.XM_015313889.1~~XM_0153…
9546   XM_015313434.1.p1  XM_015313434.1.p1 GENE.XM_015313434.1~~XM_0153…
10591  XM_015315409.1.p1  XM_015315409.1.p1 GENE.XM_015315409.1~~XM_0153…
6528   XM_006366956.2.p1  XM_006366956.2.p1 GENE.XM_006366956.2~~XM_0063…
8250   XM_015305885.1.p1  XM_015305885.1.p1 GENE.XM_015305885.1~~XM_0153…

       protein_length  probabilidade_resistencia
7738             1271                   0.991124
7737             1271                   0.991124
1986             1646                   0.990740
8115             1169                   0.990429
8116             1169                   0.990429
…                   …                          …
9816              775                   0.866533
9546              589                   0.865290
10591            1113                   0.862263
6528              761                   0.861031
8250              543                   0.859441

[229 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 223
KINASE2 : 152
GLPL : 216
MHD : 226
RNBS_A : 0
RNBS_B : 193
RNBS_C : 218
RNBS_D : 228
HRD : 2
DFG : 50
LRR1 : 71
LRR2 : 106
```

```
Finalizado predição TUBEROSUM: 15:56:04

Tempo decorrido: 0:16:09.700875
```

[28]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
 ↪rf_model,"RF",output="predicoes_physalis_rf.csv",planta="Physalis peruviana")
fim = datetime.now()
print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_rf.csv")
```

```
Predição para PHYSALIS



Inciando predição PHYSALIS:  15:56:04

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/physalis_transcriptoma.fasta, MODELO: RF
                       id                                                descricao  \
711    IABH01002505.1.p1  IABH01002505.1.p1 GENE.IABH01002505.1~~IABH010…
710    IABH01002504.1.p1  IABH01002504.1.p1 GENE.IABH01002504.1~~IABH010…
9609   IABH01022911.1.p1  IABH01022911.1.p1 GENE.IABH01022911.1~~IABH010…
9617   IABH01022919.1.p1  IABH01022919.1.p1 GENE.IABH01022919.1~~IABH010…
10240  IABH01023909.1.p1  IABH01023909.1.p1 GENE.IABH01023909.1~~IABH010…
1483   IABH01005634.1.p1  IABH01005634.1.p1 GENE.IABH01005634.1~~IABH010…
4194   IABH01013451.1.p1  IABH01013451.1.p1 GENE.IABH01013451.1~~IABH010…
4958   IABH01014862.1.p1  IABH01014862.1.p1 GENE.IABH01014862.1~~IABH010…
4957   IABH01014861.1.p1  IABH01014861.1.p1 GENE.IABH01014861.1~~IABH010…
10938  IABH01029912.1.p1  IABH01029912.1.p1 GENE.IABH01029912.1~~IABH010…
7797   IABH01019816.1.p1  IABH01019816.1.p1 GENE.IABH01019816.1~~IABH010…
10242  IABH01023913.1.p1  IABH01023913.1.p1 GENE.IABH01023913.1~~IABH010…
10241  IABH01023911.1.p1  IABH01023911.1.p1 GENE.IABH01023911.1~~IABH010…
10220  IABH01023861.1.p1  IABH01023861.1.p1 GENE.IABH01023861.1~~IABH010…
3828   IABH01012351.1.p1  IABH01012351.1.p1 GENE.IABH01012351.1~~IABH010…
10219  IABH01023859.1.p1  IABH01023859.1.p1 GENE.IABH01023859.1~~IABH010…
10218  IABH01023856.1.p1  IABH01023856.1.p1 GENE.IABH01023856.1~~IABH010…
9265   IABH01022284.1.p1  IABH01022284.1.p1 GENE.IABH01022284.1~~IABH010…
10950  IABH01030004.1.p1  IABH01030004.1.p1 GENE.IABH01030004.1~~IABH010…
```

9618	IABH01022921.1.p1	IABH01022921.1.p1	GENE.IABH01022921.1~~IABH010…
39	IABH01000147.1.p1	IABH01000147.1.p1	GENE.IABH01000147.1~~IABH010…
10217	IABH01023853.1.p1	IABH01023853.1.p1	GENE.IABH01023853.1~~IABH010…
1820	IABH01007074.1.p1	IABH01007074.1.p1	GENE.IABH01007074.1~~IABH010…
4477	IABH01013953.1.p1	IABH01013953.1.p1	GENE.IABH01013953.1~~IABH010…
4478	IABH01013954.1.p1	IABH01013954.1.p1	GENE.IABH01013954.1~~IABH010…
10249	IABH01023929.1.p1	IABH01023929.1.p1	GENE.IABH01023929.1~~IABH010…
9434	IABH01022586.1.p1	IABH01022586.1.p1	GENE.IABH01022586.1~~IABH010…
201	IABH01000746.1.p1	IABH01000746.1.p1	GENE.IABH01000746.1~~IABH010…
200	IABH01000745.1.p1	IABH01000745.1.p1	GENE.IABH01000745.1~~IABH010…
725	IABH01002538.1.p1	IABH01002538.1.p1	GENE.IABH01002538.1~~IABH010…
724	IABH01002537.1.p1	IABH01002537.1.p1	GENE.IABH01002537.1~~IABH010…
9483	IABH01022656.1.p1	IABH01022656.1.p1	GENE.IABH01022656.1~~IABH010…
9479	IABH01022654.1.p1	IABH01022654.1.p1	GENE.IABH01022654.1~~IABH010…
679	IABH01002433.1.p1	IABH01002433.1.p1	GENE.IABH01002433.1~~IABH010…
2687	IABH01009585.1.p1	IABH01009585.1.p1	GENE.IABH01009585.1~~IABH010…

	protein_length	probabilidade_resistencia
711	889	0.983535
710	889	0.983535
9609	879	0.982055
9617	881	0.981031
10240	849	0.980636
1483	908	0.980367
4194	875	0.980201
4958	884	0.978450
4957	884	0.978450
10938	890	0.978069
7797	881	0.975653
10242	852	0.973649
10241	852	0.973649
10220	865	0.965808
3828	952	0.965412
10219	870	0.964938
10218	866	0.964213
9265	960	0.960585
10950	888	0.955868
9618	873	0.953693
39	735	0.952737
10217	726	0.949760
1820	892	0.948197
4477	864	0.946099
4478	864	0.946099
10249	530	0.945611
9434	903	0.931235
201	891	0.917805
200	891	0.917805
725	1286	0.910019

```
724             1395            0.906325
9483            1390            0.899056
9479            1343            0.892820
679             1174            0.883852
2687            1247            0.878920

VALIDAÇÃO BIOLÓGICA

P_LOOP : 34
KINASE2 : 25
GLPL : 33
MHD : 35
RNBS_A : 0
RNBS_B : 33
RNBS_C : 34
RNBS_D : 35
HRD : 0
DFG : 3
LRR1 : 6
LRR2 : 17


Finalizado predição PHYSALIS: 16:13:21

Tempo decorrido: 0:17:16.870058

Arquivo salvo:
pipelineAE/predicoes_rf.csv
```

[29]:
```
'''
Aplicação - Modelo SVM
'''
print("\n\n Predição Modelo SVM")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪svm_model,"SVM",output="predicoes_arabidopsis_svm.csv",planta="Arabidopsis␣
 ↪thaliana")
fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
 Predição Modelo SVM

Predição para ARABIDOPSIS
```

```
Inciando predição ARABIDOPSIS:  16:13:21

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/arabidopsis_transcriptoma.fasta, MODELO: SVM
                     id                                                 descricao  \
4074  NM_001344104.1.p1  NM_001344104.1.p1 GENE.NM_001344104.1~~NM_0013…
8895     NM_122936.5.p1  NM_122936.5.p1 GENE.NM_122936.5~~NM_122936.5.p…
4358  NM_001344485.1.p1  NM_001344485.1.p1 GENE.NM_001344485.1~~NM_0013…
4359  NM_001344486.1.p1  NM_001344486.1.p1 GENE.NM_001344486.1~~NM_0013…
9348     NM_124096.4.p1  NM_124096.4.p1 GENE.NM_124096.4~~NM_124096.4.p…
…                   …                                                  …
7813     NM_120487.7.p1  NM_120487.7.p1 GENE.NM_120487.7~~NM_120487.7.p…
4465  NM_001344603.1.p1  NM_001344603.1.p1 GENE.NM_001344603.1~~NM_0013…
9303     NM_123996.6.p1  NM_123996.6.p1 GENE.NM_123996.6~~NM_123996.6.p…
4513  NM_001344667.1.p1  NM_001344667.1.p1 GENE.NM_001344667.1~~NM_0013…
4457  NM_001344596.1.p1  NM_001344596.1.p1 GENE.NM_001344596.1~~NM_0013…

      protein_length  probabilidade_resistencia
4074             902                   1.000000
8895             902                   1.000000
4358             909                   0.999999
4359             909                   0.999999
9348             844                   0.999999
…                …                          …
7813             601                   0.868412
4465             812                   0.863136
9303            1140                   0.862720
4513            1140                   0.862720
4457            1110                   0.855439

[199 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 174
KINASE2 : 27
GLPL : 162
MHD : 145
RNBS_A : 0
RNBS_B : 180
RNBS_C : 198
RNBS_D : 195
HRD : 6
```

```
DFG : 44
LRR1 : 44
LRR2 : 109


Finalizado predição ARABIDOPSIS: 16:13:28

Tempo decorrido: 0:00:07.317179
```

[30]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
 ↪svm_model,'SVM',output="predicoes_melongena_svm.csv",planta="Solanum␣
 ↪melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para MELONGENA



Inciando predição MELONGENA:  16:13:28

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/melongena_transcriptoma.fasta, MODELO: SVM
                      id                                                 descricao  \
274   GBEF01001042.1.p1  GBEF01001042.1.p1 GENE.GBEF01001042.1~~GBEF010…
4488  GBEF01026076.1.p1  GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010…
4489  GBEF01026077.1.p1  GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010…
3221  GBEF01023314.1.p1  GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010…
3220  GBEF01023313.1.p1  GBEF01023313.1.p1 GENE.GBEF01023313.1~~GBEF010…
3223  GBEF01023316.1.p1  GBEF01023316.1.p1 GENE.GBEF01023316.1~~GBEF010…
9160  GBEF01044249.1.p1  GBEF01044249.1.p1 GENE.GBEF01044249.1~~GBEF010…
4337  GBEF01025817.1.p1  GBEF01025817.1.p1 GENE.GBEF01025817.1~~GBEF010…
4338  GBEF01025818.1.p1  GBEF01025818.1.p1 GENE.GBEF01025818.1~~GBEF010…
3222  GBEF01023315.1.p1  GBEF01023315.1.p1 GENE.GBEF01023315.1~~GBEF010…
789   GBEF01005143.1.p1  GBEF01005143.1.p1 GENE.GBEF01005143.1~~GBEF010…
9162  GBEF01044251.1.p1  GBEF01044251.1.p1 GENE.GBEF01044251.1~~GBEF010…
9161  GBEF01044250.1.p1  GBEF01044250.1.p1 GENE.GBEF01044250.1~~GBEF010…
2372  GBEF01017801.1.p1  GBEF01017801.1.p1 GENE.GBEF01017801.1~~GBEF010…
9158  GBEF01044247.1.p1  GBEF01044247.1.p1 GENE.GBEF01044247.1~~GBEF010…
9238  GBEF01044458.1.p1  GBEF01044458.1.p1 GENE.GBEF01044458.1~~GBEF010…
7023  GBEF01034428.1.p1  GBEF01034428.1.p1 GENE.GBEF01034428.1~~GBEF010…
```

9163	GBEF01044252.1.p1	GBEF01044252.1.p1	GENE.GBEF01044252.1~~GBEF010…
7022	GBEF01034427.1.p1	GBEF01034427.1.p1	GENE.GBEF01034427.1~~GBEF010…
9159	GBEF01044248.1.p1	GBEF01044248.1.p1	GENE.GBEF01044248.1~~GBEF010…
1608	GBEF01016312.1.p1	GBEF01016312.1.p1	GENE.GBEF01016312.1~~GBEF010…
4586	GBEF01026239.1.p2	GBEF01026239.1.p2	GENE.GBEF01026239.1~~GBEF010…
1845	GBEF01016702.1.p1	GBEF01016702.1.p1	GENE.GBEF01016702.1~~GBEF010…
5996	GBEF01030153.1.p1	GBEF01030153.1.p1	GENE.GBEF01030153.1~~GBEF010…
1067	GBEF01015093.1.p1	GBEF01015093.1.p1	GENE.GBEF01015093.1~~GBEF010…
7149	GBEF01034741.1.p1	GBEF01034741.1.p1	GENE.GBEF01034741.1~~GBEF010…
6005	GBEF01030217.1.p1	GBEF01030217.1.p1	GENE.GBEF01030217.1~~GBEF010…
5473	GBEF01028520.1.p1	GBEF01028520.1.p1	GENE.GBEF01028520.1~~GBEF010…
5482	GBEF01028527.1.p1	GBEF01028527.1.p1	GENE.GBEF01028527.1~~GBEF010…
5479	GBEF01028524.1.p1	GBEF01028524.1.p1	GENE.GBEF01028524.1~~GBEF010…
5484	GBEF01028528.1.p1	GBEF01028528.1.p1	GENE.GBEF01028528.1~~GBEF010…
5488	GBEF01028531.1.p1	GBEF01028531.1.p1	GENE.GBEF01028531.1~~GBEF010…
5475	GBEF01028522.1.p1	GBEF01028522.1.p1	GENE.GBEF01028522.1~~GBEF010…
5471	GBEF01028519.1.p1	GBEF01028519.1.p1	GENE.GBEF01028519.1~~GBEF010…
3997	GBEF01025059.1.p1	GBEF01025059.1.p1	GENE.GBEF01025059.1~~GBEF010…
9237	GBEF01044457.1.p1	GBEF01044457.1.p1	GENE.GBEF01044457.1~~GBEF010…
5477	GBEF01028523.1.p1	GBEF01028523.1.p1	GENE.GBEF01028523.1~~GBEF010…
5481	GBEF01028526.1.p1	GBEF01028526.1.p1	GENE.GBEF01028526.1~~GBEF010…
5489	GBEF01028532.1.p1	GBEF01028532.1.p1	GENE.GBEF01028532.1~~GBEF010…
7487	GBEF01035527.1.p2	GBEF01035527.1.p2	GENE.GBEF01035527.1~~GBEF010…
9250	GBEF01044472.1.p1	GBEF01044472.1.p1	GENE.GBEF01044472.1~~GBEF010…

	protein_length	probabilidade_resistencia
274	867	1.000000
4488	892	1.000000
4489	866	1.000000
3221	872	0.999999
3220	881	0.999994
3223	874	0.999994
9160	579	0.999991
4337	886	0.997036
4338	886	0.997036
3222	883	0.996924
789	504	0.996334
9162	532	0.995508
9161	572	0.995272
2372	611	0.993529
9158	713	0.993055
9238	851	0.990980
7023	1126	0.986961
9163	525	0.986658
7022	1128	0.984257
9159	706	0.982164
1608	865	0.980753
4586	408	0.972200

```
1845            914                 0.957418
5996            430                 0.938028
1067           1025                 0.935103
7149            417                 0.928186
6005            532                 0.910486
5473            500                 0.901320
5482            500                 0.901320
5479            500                 0.901320
5484            500                 0.901320
5488            500                 0.901320
5475            500                 0.901320
5471            500                 0.901320
3997            974                 0.896875
9237            585                 0.892987
5477            896                 0.870193
5481            896                 0.870193
5489            548                 0.865125
7487            409                 0.863829
9250            563                 0.854160

VALIDAÇÃO BIOLÓGICA

P_LOOP : 25
KINASE2 : 13
GLPL : 40
MHD : 13
RNBS_A : 0
RNBS_B : 28
RNBS_C : 36
RNBS_D : 41
HRD : 1
DFG : 11
LRR1 : 4
LRR2 : 8


Finalizado predição MELONGENA: 16:13:35

Tempo decorrido: 0:00:06.238318
```

[31]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
 ↪svm_model,"SVM",output="predicoes_tuberosum_svm.csv",planta="Solanum␣
 ↪tuberosum")
fim = datetime.now()
```

```
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para TUBEROSUM




Inciando predição TUBEROSUM:  16:13:35

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/tuberosum_transcriptoma.fasta, MODELO: SVM
                     id                                                 descricao  \
9242  XM_015312842.1.p1  XM_015312842.1.p1 GENE.XM_015312842.1~~XM_0153…
7739  XM_015304642.1.p1  XM_015304642.1.p1 GENE.XM_015304642.1~~XM_0153…
7737  XM_015304640.1.p1  XM_015304640.1.p1 GENE.XM_015304640.1~~XM_0153…
7738  XM_015304641.1.p1  XM_015304641.1.p1 GENE.XM_015304641.1~~XM_0153…
7736  XM_015304639.1.p1  XM_015304639.1.p1 GENE.XM_015304639.1~~XM_0153…
…                   …                                                  …
8215  XM_015305770.1.p2  XM_015305770.1.p2 GENE.XM_015305770.1~~XM_0153…
8209  XM_015305767.1.p2  XM_015305767.1.p2 GENE.XM_015305767.1~~XM_0153…
8211  XM_015305768.1.p2  XM_015305768.1.p2 GENE.XM_015305768.1~~XM_0153…
9266  XM_015312945.1.p1  XM_015312945.1.p1 GENE.XM_015312945.1~~XM_0153…
8217  XM_015305771.1.p2  XM_015305771.1.p2 GENE.XM_015305771.1~~XM_0153…

      protein_length  probabilidade_resistencia
9242            1216                   1.000000
7739            1267                   1.000000
7737            1271                   1.000000
7738            1271                   1.000000
7736            1023                   1.000000
…                  …                          …
8215             269                   0.861827
8209             269                   0.861827
8211             269                   0.861827
9266             542                   0.860106
8217             276                   0.851684

[355 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 289
KINASE2 : 168
GLPL : 304
MHD : 306
```

```
RNBS_A : 0
RNBS_B : 268
RNBS_C : 326
RNBS_D : 349
HRD : 3
DFG : 58
LRR1 : 85
LRR2 : 158


Finalizado predição TUBEROSUM: 16:13:42

Tempo decorrido: 0:00:07.162423
```

[32]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
 ↪svm_model,"SVM",output="predicoes_physalis_svm.csv",planta="Physalis␣
 ↪peruviana")
fim = datetime.now()
print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_svm.csv")
```

```
Predição para PHYSALIS



Inciando predição PHYSALIS:  16:13:42

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAE/physalis_transcriptoma.fasta, MODELO: SVM
                       id                                              descricao  \
10240  IABH01023909.1.p1  IABH01023909.1.p1 GENE.IABH01023909.1~~IABH010…
10242  IABH01023913.1.p1  IABH01023913.1.p1 GENE.IABH01023913.1~~IABH010…
10241  IABH01023911.1.p1  IABH01023911.1.p1 GENE.IABH01023911.1~~IABH010…
10249  IABH01023929.1.p1  IABH01023929.1.p1 GENE.IABH01023929.1~~IABH010…
9618   IABH01022921.1.p1  IABH01022921.1.p1 GENE.IABH01022921.1~~IABH010…
…                    …                                                 …
5099   IABH01015139.1.p1  IABH01015139.1.p1 GENE.IABH01015139.1~~IABH010…
10244  IABH01023915.1.p1  IABH01023915.1.p1 GENE.IABH01023915.1~~IABH010…
3160   IABH01010795.1.p1  IABH01010795.1.p1 GENE.IABH01010795.1~~IABH010…
```

```
5281   IABH01015446.1.p1  IABH01015446.1.p1 GENE.IABH01015446.1~~IABH010…
5280   IABH01015445.1.p1  IABH01015445.1.p1 GENE.IABH01015445.1~~IABH010…

       protein_length  probabilidade_resistencia
10240             849                   1.000000
10242             852                   1.000000
10241             852                   1.000000
10249             530                   1.000000
9618              873                   1.000000
…                 …                      …
5099              448                   0.882528
10244             194                   0.860510
3160              974                   0.860206
5281              730                   0.856102
5280              735                   0.853556

[82 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 49
KINASE2 : 30
GLPL : 68
MHD : 64
RNBS_A : 0
RNBS_B : 69
RNBS_C : 78
RNBS_D : 82
HRD : 0
DFG : 6
LRR1 : 7
LRR2 : 36


Finalizado predição PHYSALIS: 16:13:49

Tempo decorrido: 0:00:07.507653

Arquivo salvo:
pipelineAE/predicoes_svm.csv
```

[33]:
```
print(df_validacao)
```

```
[{'P_LOOP': 170, 'KINASE2': 27, 'GLPL': 159, 'MHD': 143, 'RNBS_A': 0, 'RNBS_B':
166, 'RNBS_C': 176, 'RNBS_D': 176, 'HRD': 6, 'DFG': 45, 'LRR1': 39, 'LRR2': 95,
'planta': 'Arabidopsis thaliana', 'modelo': 'XGBoost'}, {'P_LOOP': 15,
'KINASE2': 12, 'GLPL': 24, 'MHD': 16, 'RNBS_A': 0, 'RNBS_B': 14, 'RNBS_C': 24,
'RNBS_D': 31, 'HRD': 1, 'DFG': 1, 'LRR1': 4, 'LRR2': 14, 'planta': 'Solanum
```

```
melongena', 'modelo': 'XGBoost'}, {'P_LOOP': 280, 'KINASE2': 170, 'GLPL': 286,
'MHD': 299, 'RNBS_A': 0, 'RNBS_B': 255, 'RNBS_C': 311, 'RNBS_D': 331, 'HRD': 3,
'DFG': 55, 'LRR1': 83, 'LRR2': 147, 'planta': 'Solanum tuberosum', 'modelo':
'XGBoost'}, {'P_LOOP': 51, 'KINASE2': 30, 'GLPL': 59, 'MHD': 65, 'RNBS_A': 0,
'RNBS_B': 60, 'RNBS_C': 74, 'RNBS_D': 78, 'HRD': 1, 'DFG': 5, 'LRR1': 7, 'LRR2':
31, 'planta': 'Physalis peruviana', 'modelo': 'XGBoost'}, {'P_LOOP': 128,
'KINASE2': 26, 'GLPL': 126, 'MHD': 116, 'RNBS_A': 0, 'RNBS_B': 122, 'RNBS_C':
127, 'RNBS_D': 128, 'HRD': 6, 'DFG': 30, 'LRR1': 29, 'LRR2': 77, 'planta':
'Arabidopsis thaliana', 'modelo': 'RF'}, {'P_LOOP': 10, 'KINASE2': 10, 'GLPL':
10, 'MHD': 7, 'RNBS_A': 0, 'RNBS_B': 3, 'RNBS_C': 6, 'RNBS_D': 10, 'HRD': 0,
'DFG': 0, 'LRR1': 4, 'LRR2': 7, 'planta': 'Solanum melongena', 'modelo': 'RF'},
{'P_LOOP': 223, 'KINASE2': 152, 'GLPL': 216, 'MHD': 226, 'RNBS_A': 0, 'RNBS_B':
193, 'RNBS_C': 218, 'RNBS_D': 228, 'HRD': 2, 'DFG': 50, 'LRR1': 71, 'LRR2': 106,
'planta': 'Solanum tuberosum', 'modelo': 'RF'}, {'P_LOOP': 34, 'KINASE2': 25,
'GLPL': 33, 'MHD': 35, 'RNBS_A': 0, 'RNBS_B': 33, 'RNBS_C': 34, 'RNBS_D': 35,
'HRD': 0, 'DFG': 3, 'LRR1': 6, 'LRR2': 17, 'planta': 'Physalis peruviana',
'modelo': 'RF'}, {'P_LOOP': 174, 'KINASE2': 27, 'GLPL': 162, 'MHD': 145,
'RNBS_A': 0, 'RNBS_B': 180, 'RNBS_C': 198, 'RNBS_D': 195, 'HRD': 6, 'DFG': 44,
'LRR1': 44, 'LRR2': 109, 'planta': 'Arabidopsis thaliana', 'modelo': 'SVM'},
{'P_LOOP': 25, 'KINASE2': 13, 'GLPL': 40, 'MHD': 13, 'RNBS_A': 0, 'RNBS_B': 28,
'RNBS_C': 36, 'RNBS_D': 41, 'HRD': 1, 'DFG': 11, 'LRR1': 4, 'LRR2': 8, 'planta':
'Solanum melongena', 'modelo': 'SVM'}, {'P_LOOP': 289, 'KINASE2': 168, 'GLPL':
304, 'MHD': 306, 'RNBS_A': 0, 'RNBS_B': 268, 'RNBS_C': 326, 'RNBS_D': 349,
'HRD': 3, 'DFG': 58, 'LRR1': 85, 'LRR2': 158, 'planta': 'Solanum tuberosum',
'modelo': 'SVM'}, {'P_LOOP': 49, 'KINASE2': 30, 'GLPL': 68, 'MHD': 64, 'RNBS_A':
0, 'RNBS_B': 69, 'RNBS_C': 78, 'RNBS_D': 82, 'HRD': 0, 'DFG': 6, 'LRR1': 7,
'LRR2': 36, 'planta': 'Physalis peruviana', 'modelo': 'SVM'}]
```

[34]:
```
df_copia = df_validacao.copy()
df_inicial = pd.DataFrame(df_validacao)

df_validacao = df_inicial.melt(
    id_vars=["planta", "modelo"],
    var_name="motivo",
    value_name="contagem"
)
print(df_validacao.head())

df_validacao.to_csv(f"{OUTPUT_DIR}/df_validacao.csv", index=False)
```

```
                planta   modelo  motivo  contagem
0  Arabidopsis thaliana  XGBoost  P_LOOP       170
1     Solanum melongena  XGBoost  P_LOOP        15
2     Solanum tuberosum  XGBoost  P_LOOP       280
3    Physalis peruviana  XGBoost  P_LOOP        51
4  Arabidopsis thaliana       RF  P_LOOP       128
```

[35]:
```
#Scatterplot
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="planta",
    style="modelo",
    s=200
)
# plt.title( "Validação biológica dos motivos conservados")
plt.ylabel("Número de ocorrências")
plt.xlabel("Motivo")
plt.legend(
    bbox_to_anchor=(1.05,1),
    loc="upper left"
)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/Validacao_biologica_dos_motivos_conservados.png")
plt.close()
# plt.show()

#Bubble Chart
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="modelo",
    hue="planta",
    size="contagem",
    sizes=(50,600)
)
# plt.title("Distribuição dos motivos por modelo e espécie")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/distribuicao_dos_motivos_por_modelo_especie.png")
plt.close()
# plt.show()


#Facetas por planta
g = sns.catplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="modelo",
    col="planta",
    kind="bar",
```

```
    height=5,
    aspect=1.2
)
g.set_xticklabels(rotation=45)
plt.savefig(f"{OUTPUT_DIR}/facetas_por_planta.png")
plt.close()
# plt.show()

#por planta

#Physalis peruviana
df_physalis = df_validacao[df_validacao["planta"] == "Physalis peruviana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_physalis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Physalis peruviana", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_physalis.png", dpi=300)
plt.close()

#Solanum melongena
df_melongena = df_validacao[df_validacao["planta"] == "Solanum melongena"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_melongena,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
```

```
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum melongena", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_melongena.png", dpi=300)
plt.close()

#Solanum tuberosum
df_tuberosum = df_validacao[df_validacao["planta"] == "Solanum tuberosum"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_tuberosum,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum tuberosum", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_tuberosum.png", dpi=300)
plt.close()

#Arabidopsis thaliana
df_arabidopsis = df_validacao[df_validacao["planta"] == "Arabidopsis thaliana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_arabidopsis,
```

```
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Arabidopsis thaliana", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_arabidopsis.png", dpi=300)
plt.close()

#Heatmap
df_validacao["grupo"] = (
    df_validacao["planta"]
    + "_"
    + df_validacao["modelo"]
)

pivot = df_validacao.pivot_table(
    index="motivo",
    columns="grupo",
    values="contagem",
    fill_value=0
)


plt.figure(figsize=(14, 8))
sns.heatmap(
    pivot,
    annot=True,
    fmt="g",
    cmap="YlGnBu",
    linewidths=0.5,
    cbar_kws={'label': 'Contagem'} # Nomeia a barra de cores lateral
)

# plt.title("Distribuição dos Motivos Conservados por Planta e Modelo",␣
 ↪fontsize=14, fontweight="bold", pad=15)
```

```
plt.ylabel("Motivo", fontsize=12)
plt.xlabel("Grupo (Planta & Modelo)", fontsize=12)
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/heatmap_distribuicao_dos_motivos_conservados.
  ↪png",dpi=300)
plt.close()
# plt.show()


print("========= Fim do Pipeline ============")
```

```
========= Fim do Pipeline ============
```